```python
import pandas as pd
df = pd.read_csv("/content/WA_Fn-UseC_-Telco-Customer-Churn.csv")
print (df)
```

```
      customerID  gender  SeniorCitizen Partner Dependents  tenure  \
0     7590-VHVEG  Female              0     Yes         No       1
1     5575-GNVDE    Male              0      No         No      34
2     3668-QPYBK    Male              0      No         No       2
3     7795-CFOCW    Male              0      No         No      45
4     9237-HQITU  Female              0      No         No       2
...          ...     ...            ...     ...        ...     ...
7038  6840-RESVB    Male              0     Yes        Yes      24
7039  2234-XADUH  Female              0     Yes        Yes      72
7040  4801-JZAZL  Female              0     Yes        Yes      11
7041  8361-LTMKD    Male              1     Yes         No       4
7042  3186-AJIEK    Male              0      No         No      66

     PhoneService     MultipleLines InternetService
OnlineSecurity  ...  \
0              No  No phone service             DSL
No  ...
1             Yes                No             DSL
Yes  ...
2             Yes                No             DSL
Yes  ...
3              No  No phone service             DSL
Yes  ...
4             Yes                No     Fiber optic
No  ...
...           ...               ...             ...             ...  ..
.
7038          Yes               Yes             DSL
Yes  ...
7039          Yes               Yes     Fiber optic
No  ...
7040           No  No phone service             DSL
Yes  ...
7041          Yes               Yes     Fiber optic
No  ...
7042          Yes                No     Fiber optic
Yes  ...

     DeviceProtection TechSupport StreamingTV StreamingMovies
Contract  \
0                  No          No          No               No  Month-
to-month
1                 Yes          No          No               No
One year
2                  No          No          No               No  Month-
to-month
```

```
3                  Yes         Yes          No              No
One year
4                   No          No          No              No  Month-
to-month
...                ...         ...         ...             ...
...
7038               Yes         Yes         Yes             Yes
One year
7039               Yes          No         Yes             Yes
One year
7040                No          No          No              No  Month-
to-month
7041                No          No          No              No  Month-
to-month
7042               Yes         Yes         Yes             Yes
Two year

     PaperlessBilling              PaymentMethod MonthlyCharges
TotalCharges  \
0                  Yes           Electronic check          29.85
29.85
1                   No               Mailed check          56.95
1889.5
2                  Yes               Mailed check          53.85
108.15
3                   No  Bank transfer (automatic)          42.30
1840.75
4                  Yes           Electronic check          70.70
151.65
...                ...                        ...            ...
...
7038               Yes               Mailed check          84.80
1990.5
7039               Yes    Credit card (automatic)         103.20
7362.9
7040               Yes           Electronic check          29.60
346.45
7041               Yes               Mailed check          74.40
306.6
7042               Yes  Bank transfer (automatic)         105.65
6844.5

     Churn
0       No
1       No
2      Yes
3       No
4      Yes
...    ...
```

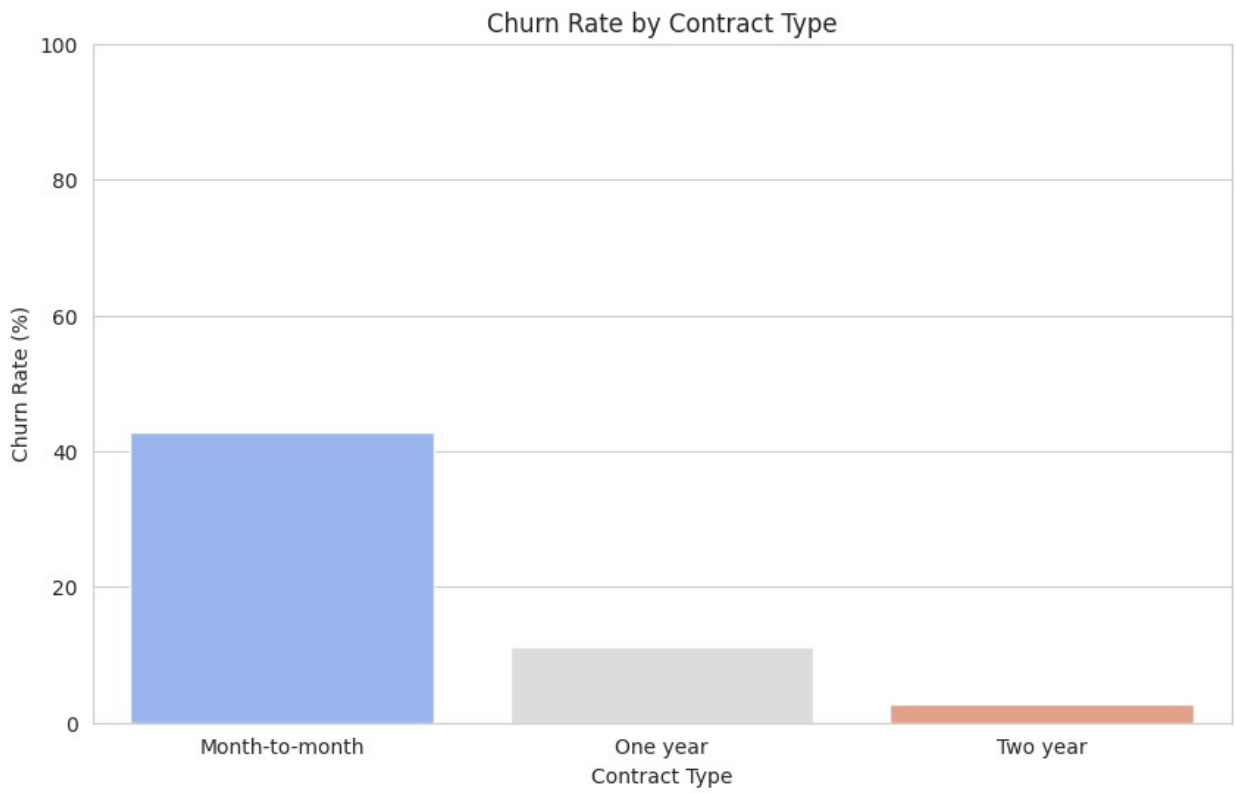
```
7038    No
7039    No
7040    No
7041   Yes
7042    No

[7043 rows x 21 columns]
```

```
# Calculate churn rate by contract type
contract_churn = df.groupby(['Contract',
'Churn']).size().unstack(fill_value=0)
contract_churn['Total'] = contract_churn['Yes'] + contract_churn['No']
contract_churn['ChurnRate'] = (contract_churn['Yes'] /
contract_churn['Total']) * 100

print("Churn Rate by Contract Type:")
display(contract_churn)
```

```
Churn Rate by Contract Type:
```


```
{"summary":"{\n  \"name\": \"contract_churn\",\n  \"rows\": 3,\n
\"fields\": [\n    {\n      \"column\": \"Contract\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 3,\n        \"samples\": [\n          \"Month-
to-month\",\n          \"One year\",\n          \"Two year\"\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"No\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 461,\n        \"min\": 1307,\n        \"max\": 2220,\n
\"num_unique_values\": 3,\n        \"samples\": [\n          2220,\n
1307,\n          1647\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Yes\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 895,\n        \"min\": 48,\n        \"max\": 1655,\n
\"num_unique_values\": 3,\n        \"samples\": [\n          1655,\n
166,\n          48\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Total\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 1327,\n        \"min\": 1473,\n        \"max\": 3875,\n
\"num_unique_values\": 3,\n        \"samples\": [\n          3875,\n
1473,\n          1695\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"ChurnRate\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 21.015542555187874,\n        \"min\":
2.831858407079646,\n        \"max\": 42.70967741935484,\n
\"num_unique_values\": 3,\n        \"samples\": [\n
42.70967741935484,\n          11.269517990495586,\n
2.831858407079646\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe","variable_name":"contract_churn"}
```

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 6))
sns.barplot(x=contract_churn.index, y='ChurnRate',
data=contract_churn, palette='coolwarm', hue=contract_churn.index,
legend=False)
plt.title('Churn Rate by Contract Type')
plt.xlabel('Contract Type')
plt.ylabel('Churn Rate (%)')
plt.ylim(0, 100)
plt.show()
```



Analysis of Churn Rate by Contract Type

This bar chart clearly illustrates the churn rate across different contract types:

- **Month-to-month contracts** exhibit the highest churn rate, indicating that customers on these plans are most likely to leave.
- **One-year contracts** have a significantly lower churn rate compared to month-to-month.
- **Two-year contracts** show the lowest churn rate, suggesting that customers committed to longer-term contracts are much more loyal. This might be due to incentives offered for longer contracts or a higher satisfaction level among these customers.

Retention Rate by Contract Type

Building on the churn rate analysis, let's visualize the retention rate across different contract types. Retention rate is simply (100% - Churn Rate).

Customer Lifetime Value (CLV) Calculation

To calculate a simplified Customer Lifetime Value (CLV), we can multiply a customer's `MonthlyCharges` by their `tenure` (in months). This gives us an estimate of the total revenue a customer has generated or is expected to generate during their relationship with the company.

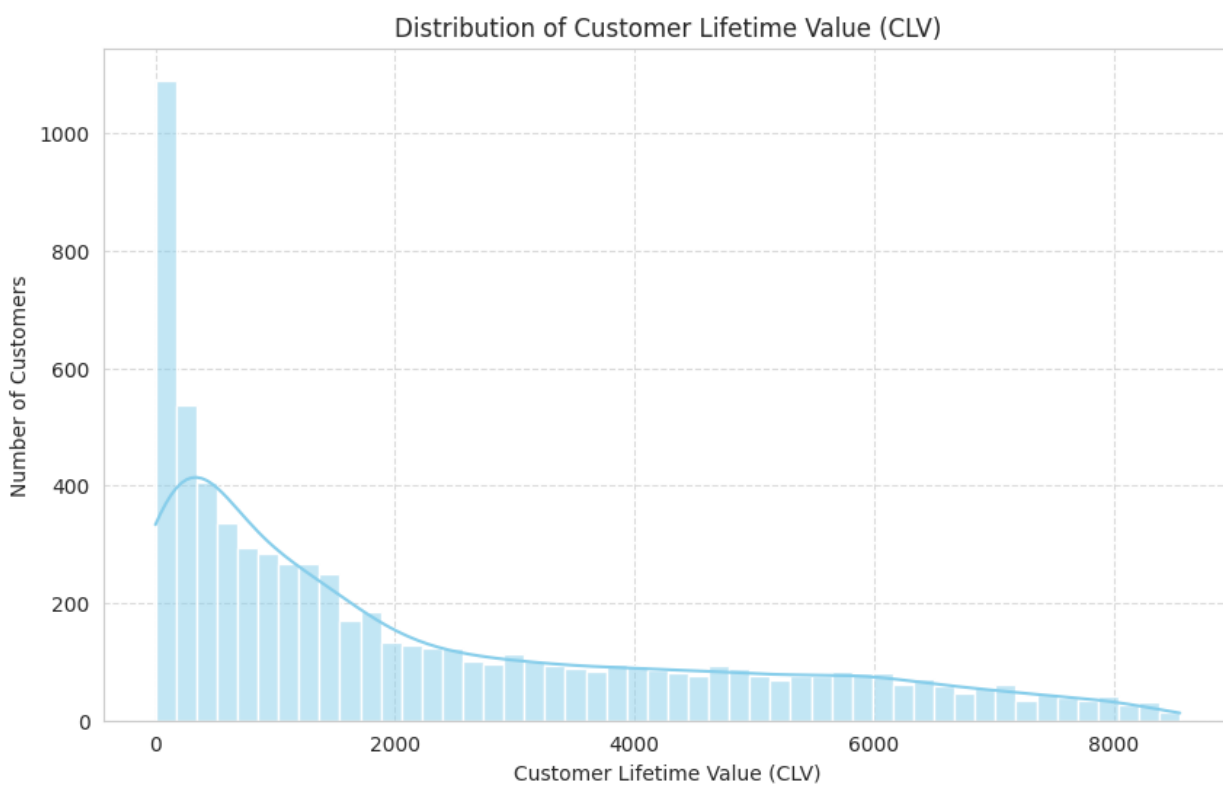
```
# Calculate a simplified Customer Lifetime Value (CLV)
# CLV = MonthlyCharges * tenure
df['CLV'] = df['MonthlyCharges'] * df['tenure']

print("Descriptive Statistics for Customer Lifetime Value (CLV):")
display(df['CLV'].describe())

print("Top 10 Customers by CLV:")
display(df[['customerID', 'tenure', 'MonthlyCharges', 'TotalCharges',
'CLV']].sort_values(by='CLV', ascending=False).head(10))
```

```
Descriptive Statistics for Customer Lifetime Value (CLV):

count    7043.000000
mean     2279.581350
std      2264.729447
min         0.000000
25%       394.000000
50%      1393.600000
75%      3786.100000
max      8550.000000
Name: CLV, dtype: float64

Top 10 Customers by CLV:
```

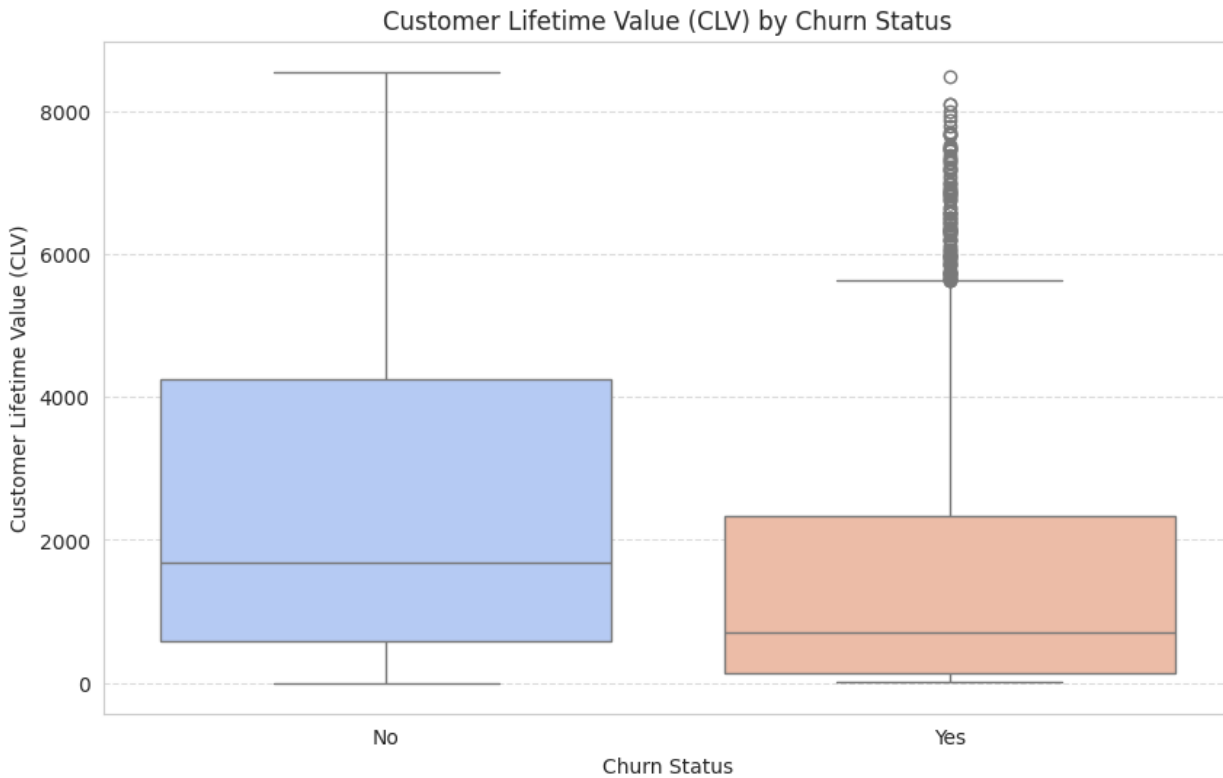

{"summary":"{\n \"name\": \"display(df[['customerID', 'tenure', 'MonthlyCharges', 'TotalCharges', 'CLV']]\",\n \"rows\": 10,\n \"fields\": [\n {\n \"column\": \"customerID\",\n \"properties\": {\n \"dtype\": \"string\",\n \"num_unique_values\": 10,\n \"samples\": [\n \"9788-HNGUT\",\n \"9924-JPRMC\",\n \"6904-JLBGY\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"tenure\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0,\n \"min\": 71,\n \"max\": 72,\n \"num_unique_values\": 2,\n \"samples\": [\n 71,\n 72\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"MonthlyCharges\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\":

0.6684143758012537,\n \"min\": 116.85,\n \"max\": 118.75,\n \"num_unique_values\": 10,\n \"samples\": [\n 116.95,\n 118.2\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"TotalCharges\",\n \"properties\": {\n \"dtype\": \"string\",\n \"num_unique_values\": 10,\n \"samples\": [\n \"8594.4\",\n \"8547.15\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"CLV\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 43.386982494753084,\n \"min\": 8413.199999999999,\n \"max\": 8550.0,\n \"num_unique_values\": 10,\n \"samples\": [\n 8420.4,\n 8510.4\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n }\n]\n}","type":"dataframe"}

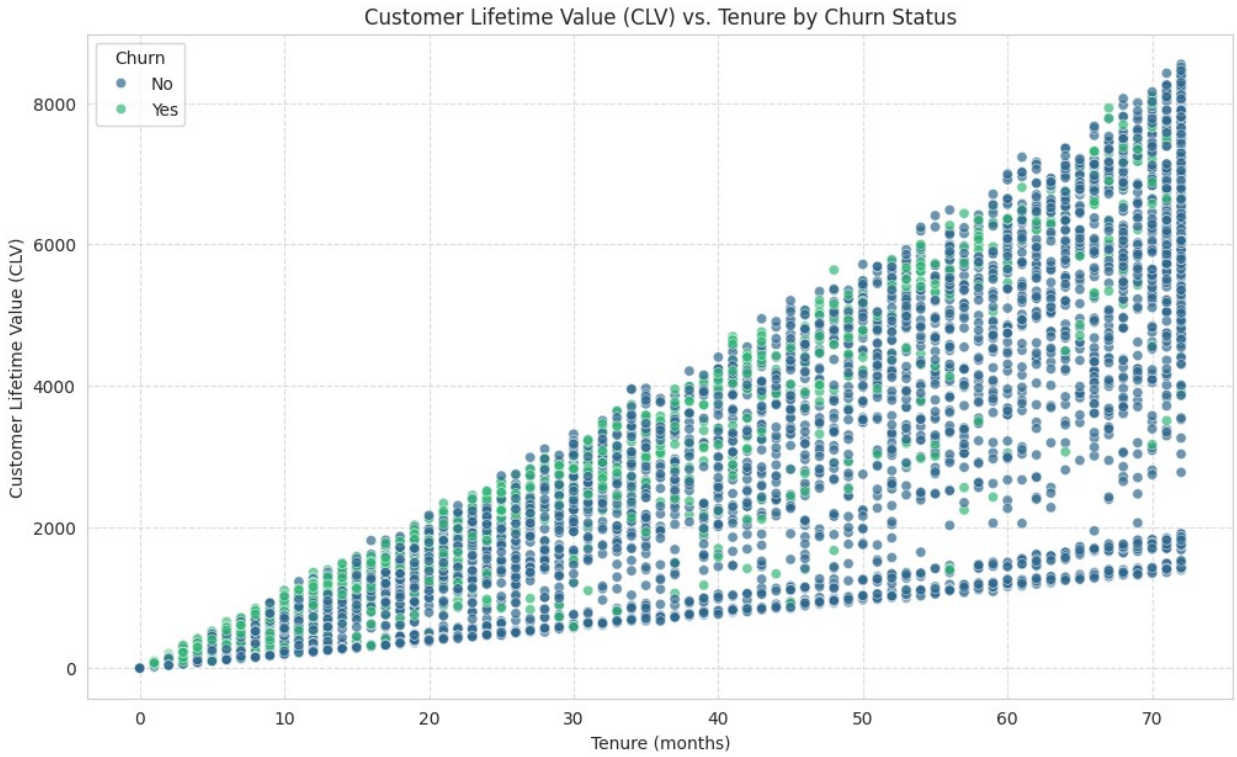
```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 6))
sns.histplot(df['CLV'], bins=50, kde=True, color='skyblue')
plt.title('Distribution of Customer Lifetime Value (CLV)')
plt.xlabel('Customer Lifetime Value (CLV)')
plt.ylabel('Number of Customers')
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```



```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 6))
sns.boxplot(x='Churn', y='CLV', data=df, palette='coolwarm',
hue='Churn', legend=False)
plt.title('Customer Lifetime Value (CLV) by Churn Status')
plt.xlabel('Churn Status')
plt.ylabel('Customer Lifetime Value (CLV)')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```




```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12, 7))
sns.scatterplot(data=df, x='tenure', y='CLV', hue='Churn',
palette='viridis', alpha=0.7)
plt.title('Customer Lifetime Value (CLV) vs. Tenure by Churn Status')
plt.xlabel('Tenure (months)')
plt.ylabel('Customer Lifetime Value (CLV)')
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```



Churn Rate by CLV Segment

Let's segment customers based on their Customer Lifetime Value (CLV) to understand how churn varies across different CLV tiers. This will help identify if high-value customers are less likely to churn, or if there are specific CLV ranges that are more prone to churn.

```
# Define the number of CLV segments and their labels
num_clv_segments = 3
clv_segment_labels = ['Low CLV', 'Medium CLV', 'High CLV']

# Create CLV segments using pd.qcut (quantile-based)
df['CLVSegment'] = pd.qcut(df['CLV'], q=num_clv_segments,
labels=clv_segment_labels, precision=2, duplicates='drop')

print("Distribution of customers across CLV Segments:")
display(df['CLVSegment'].value_counts().sort_index())

# Calculate churn rate for each CLV segment
clv_churn = df.groupby(['CLVSegment', 'Churn'],
observed=False).size().unstack(fill_value=0)
clv_churn['Total'] = clv_churn['Yes'] + clv_churn['No']
clv_churn['ChurnRate'] = (clv_churn['Yes'] / clv_churn['Total']) * 100

print("\nChurn Rate by CLV Segment:")
display(clv_churn)
```

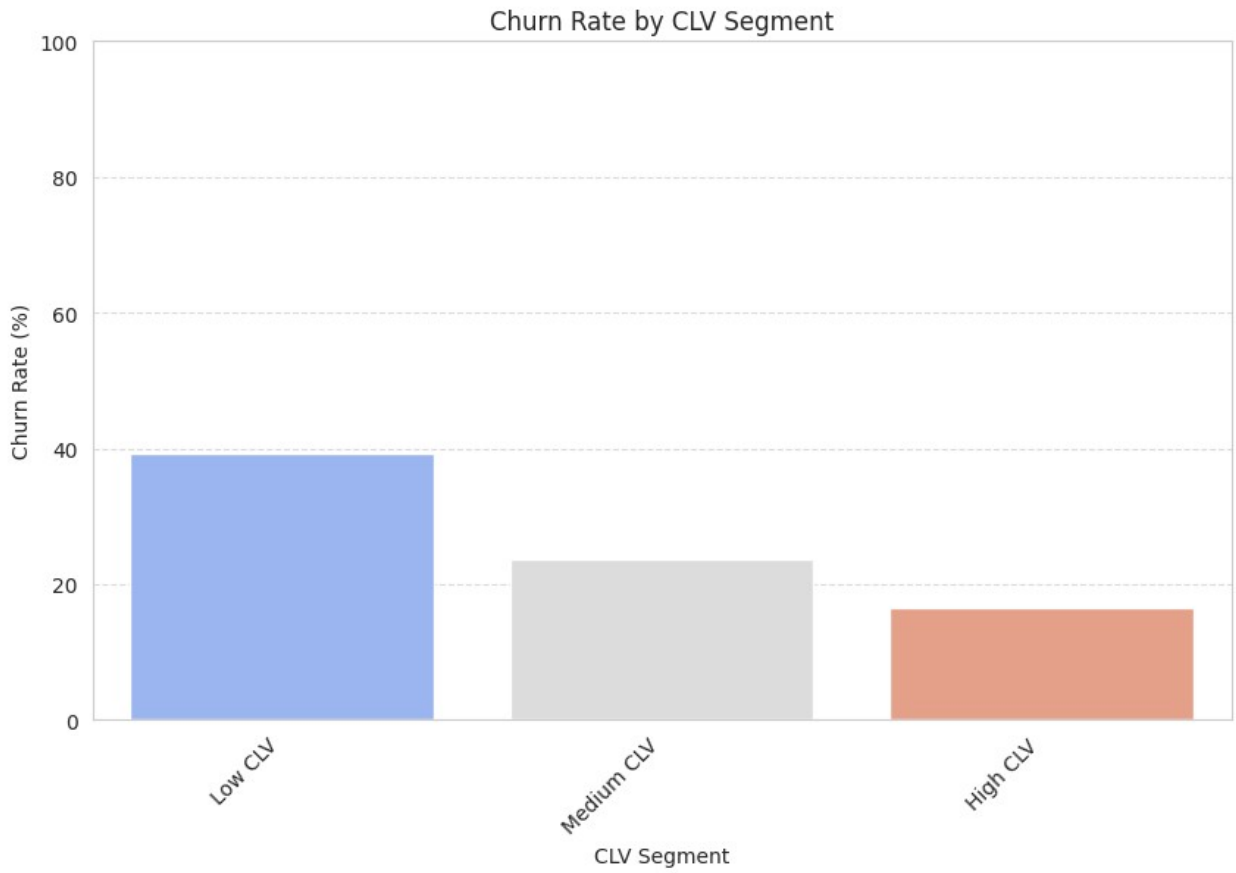
```
Distribution of customers across CLV Segments:
```

```
CLVSegment
Low CLV       2349
Medium CLV    2346
High CLV      2348
Name: count, dtype: int64


Churn Rate by CLV Segment:
```

{"summary":"{\n \"name\": \"clv_churn\",\n \"rows\": 3,\n \"fields\": [\n {\n \"column\": \"CLVSegment\",\n \"properties\": {\n \"dtype\": \"category\",\n \"num_unique_values\": 3,\n \"samples\": [\n \"Low CLV\",\n \"Medium CLV\",\n \"High CLV\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"No\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 271,\n \"min\": 1427,\n \"max\": 1959,\n \"num_unique_values\": 3,\n \"samples\": [\n 1427,\n 1788,\n 1959\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Yes\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 272,\n \"min\": 389,\n \"max\": 922,\n \"num_unique_values\": 3,\n \"samples\": [\n 922,\n 558,\n 389\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Total\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 1,\n \"min\": 2346,\n \"max\": 2349,\n \"num_unique_values\": 3,\n \"samples\": [\n 2349,\n 2346,\n 2348\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"ChurnRate\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 11.588938087095313,\n \"min\": 16.567291311754683,\n \"max\": 39.250744997871436,\n \"num_unique_values\": 3,\n \"samples\": [\n 39.250744997871436,\n 23.785166240409207,\n 16.567291311754683\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n }\n]\n}","type":"dataframe","variable_name":"clv_churn"}

```python
import matplotlib.pyplot as plt
import seaborn as sns

# Visualize churn rate by CLV segment
plt.figure(figsize=(10, 6))
sns.barplot(x=clv_churn.index, y='ChurnRate', data=clv_churn,
palette='coolwarm', hue=clv_churn.index, legend=False)
plt.title('Churn Rate by CLV Segment')
plt.xlabel('CLV Segment')
plt.ylabel('Churn Rate (%)')
```

```
plt.ylim(0, 100)
plt.xticks(rotation=45, ha='right')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

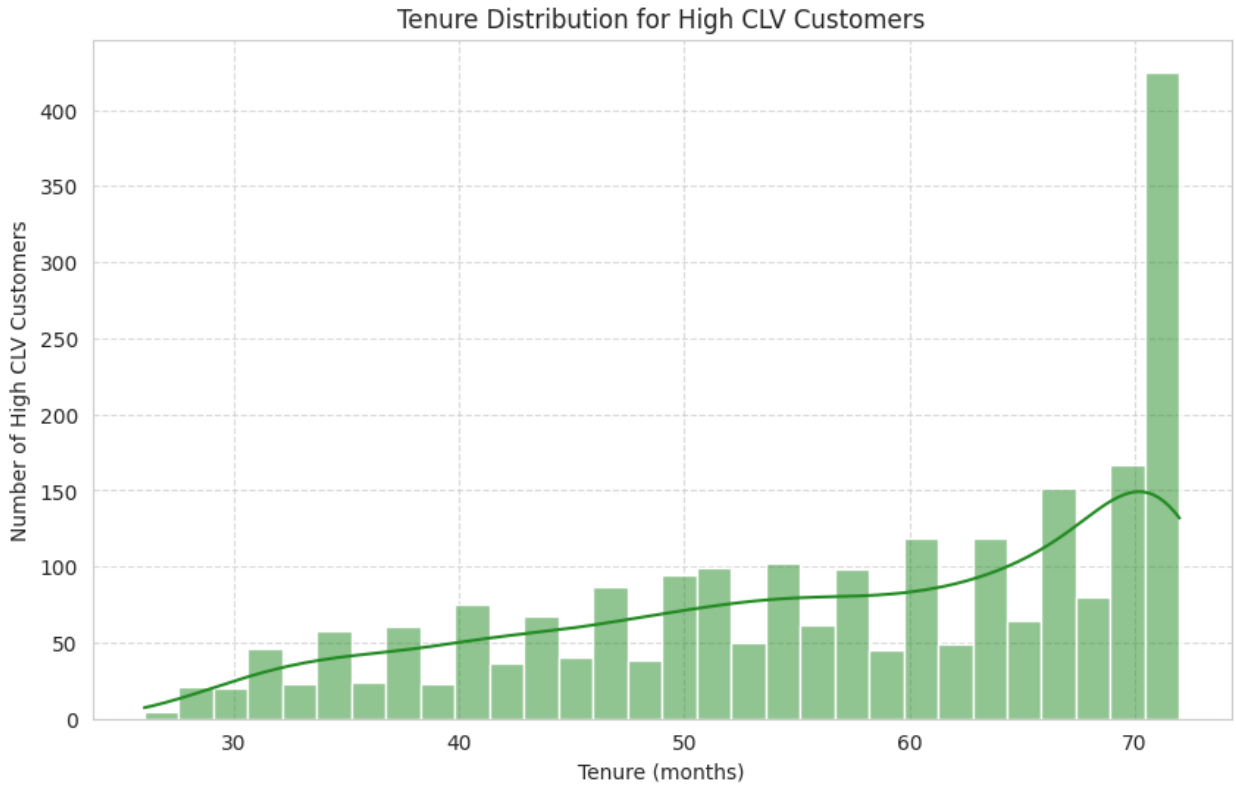


Tenure Distribution for High CLV Customers

To understand the characteristics of our most valuable customers, let's examine the distribution of 'tenure' specifically for those in the 'High CLV' segment. This can provide insights into how long these high-value customers typically stay with the service.

```
# Filter the DataFrame for High CLV customers
high_clv_customers = df[df['CLVSegment'] == 'High CLV']

# Create a histogram to visualize the distribution of 'tenure' for
High CLV customers
plt.figure(figsize=(10, 6))
sns.histplot(high_clv_customers['tenure'], bins=30, kde=True,
color='forestgreen')
plt.title('Tenure Distribution for High CLV Customers')
plt.xlabel('Tenure (months)')
plt.ylabel('Number of High CLV Customers')
```

```
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```



Estimated Revenue Lost Due to Churn

To quantify the financial impact of churn, we can estimate the total revenue lost by summing the calculated Customer Lifetime Value (CLV) for all customers who have churned.

```
# Filter for churned customers
churned_customers_df = df[df['Churn'] == 'Yes']

# Calculate the total CLV for churned customers
total_revenue_lost = churned_customers_df['CLV'].sum()

print(f"Estimated Total Revenue Lost Due to Churn: $
{total_revenue_lost:,.2f}")
```

```
Estimated Total Revenue Lost Due to Churn: $2,862,576.90
```

Potential Revenue Savings by Reducing Churn in High CLV Segment

To estimate potential revenue savings, we'll focus on the 'High CLV' segment and calculate the CLV lost from customers who churned in this segment. Then, we can project savings based on a hypothetical churn reduction percentage.

```
# Filter for churned customers in the 'High CLV' segment
churned_high_clv_customers = df[(df['CLVSegment'] == 'High CLV') &
(df['Churn'] == 'Yes')]

# Calculate the total CLV lost from these churned high CLV customers
total_clv_lost_high_clv = churned_high_clv_customers['CLV'].sum()

print(f"Total CLV lost from churned High CLV customers: $
{total_clv_lost_high_clv:,.2f}")

# Define a hypothetical churn reduction percentage for the High CLV
segment
# For example, if we aim to reduce churn in this segment by 10%
churn_reduction_percentage = 0.25 # 25%

potential_revenue_savings = total_clv_lost_high_clv *
churn_reduction_percentage

print(f"Potential revenue savings by reducing churn in High CLV
segment by {churn_reduction_percentage*100}%: $
{potential_revenue_savings:,.2f}")
print("You can adjust the 'churn_reduction_percentage' variable above
to explore different scenarios.")
```

```
Total CLV lost from churned High CLV customers: $1,837,567.95
Potential revenue savings by reducing churn in High CLV segment by
25.0%: $459,391.99
You can adjust the 'churn_reduction_percentage' variable above to
explore different scenarios.
```

Comparison of Potential Revenue Savings for Different Churn Reduction Percentages

```
# Define the first hypothetical churn reduction percentage
churn_reduction_percentage_15 = 0.15 # 15%
potential_revenue_savings_15 = total_clv_lost_high_clv *
churn_reduction_percentage_15
print(f"Potential revenue savings by reducing churn in High CLV
segment by {churn_reduction_percentage_15*100}%: $
{potential_revenue_savings_15:,.2f}")

# Define the second hypothetical churn reduction percentage
churn_reduction_percentage_50 = 0.50 # 50%
potential_revenue_savings_50 = total_clv_lost_high_clv *
churn_reduction_percentage_50
print(f"Potential revenue savings by reducing churn in High CLV
segment by {churn_reduction_percentage_50*100}%: $
{potential_revenue_savings_50:,.2f}")
```

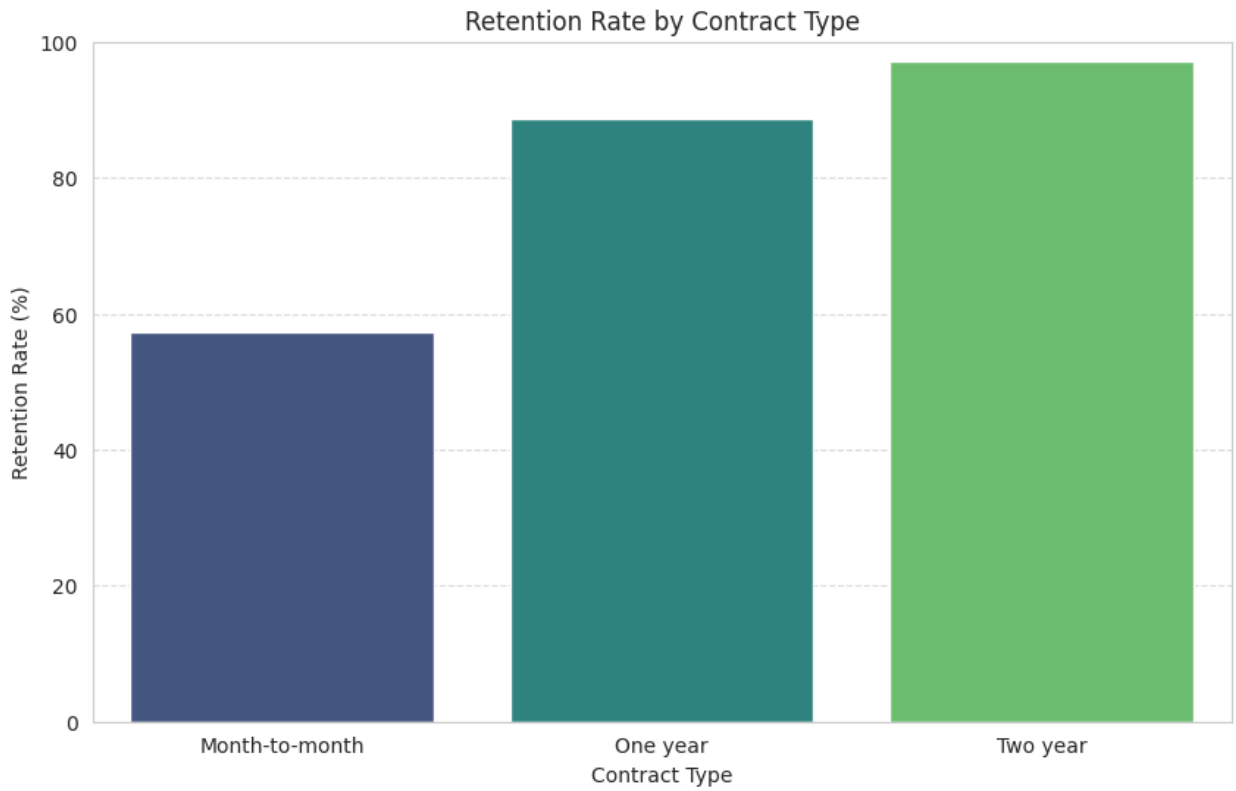
```
Potential revenue savings by reducing churn in High CLV segment by
15.0%: $275,635.19
Potential revenue savings by reducing churn in High CLV segment by
50.0%: $918,783.98
```

```
# Calculate Retention Rate
contract_churn['RetentionRate'] = 100 - contract_churn['ChurnRate']

print("Retention Rate by Contract Type:")
display(contract_churn[['Total', 'RetentionRate']])
```

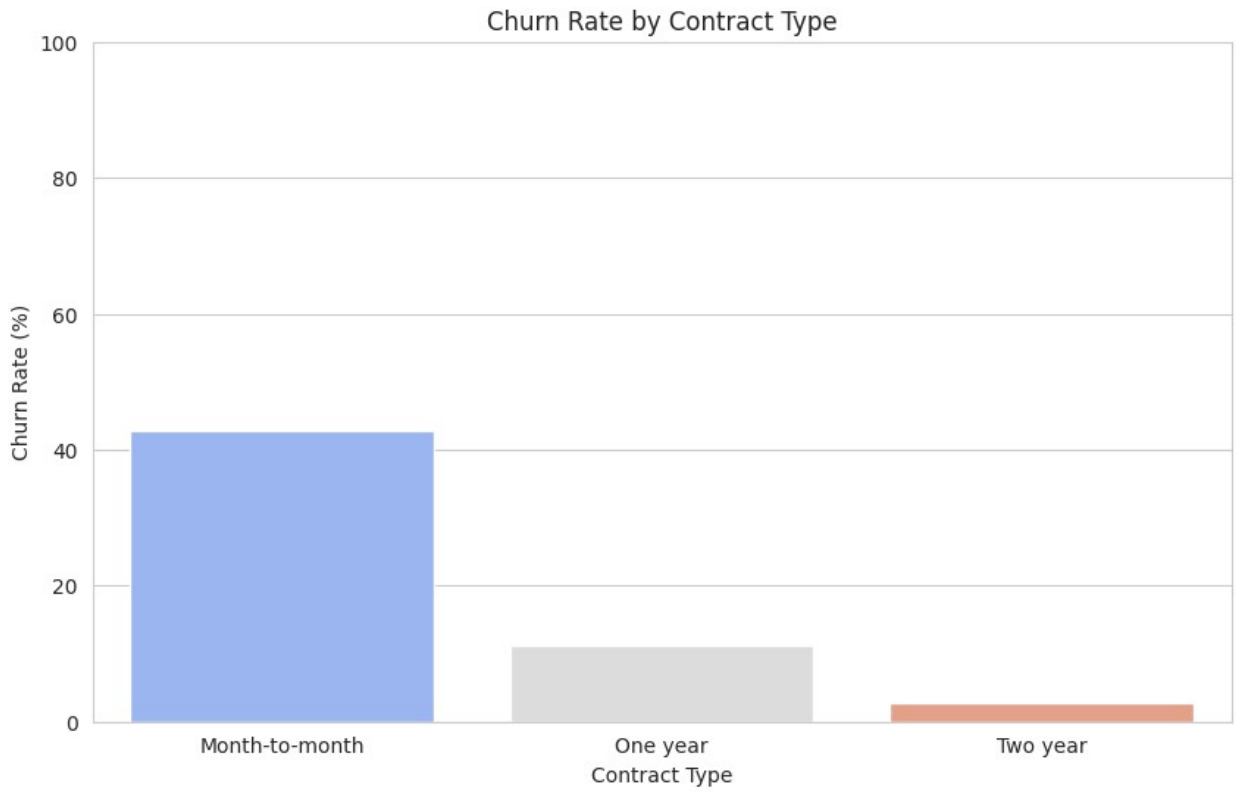
```
Retention Rate by Contract Type:
```


```
{"summary":"{\n  \"name\": \"display(contract_churn[['Total',
'RetentionRate']])\",\n  \"rows\": 3,\n  \"fields\": [\n  {\n
\"column\": \"Contract\",\n  \"properties\": {\n  \"dtype\":
\"string\",\n  \"num_unique_values\": 3,\n  \"samples\":
[\n  \"Month-to-month\",\n  \"One year\",\n
\"Two year\"\n  ],\n  \"semantic_type\": \"\",\n
\"description\": \"\"\n  }\n  },\n  {\n  \"column\":
\"Total\",\n  \"properties\": {\n  \"dtype\": \"number\",\n
\"std\": 1327,\n  \"min\": 1473,\n  \"max\": 3875,\n
\"num_unique_values\": 3,\n  \"samples\": [\n  3875,\n
1473,\n  1695\n  ],\n  \"semantic_type\": \"\",\n
\"description\": \"\"\n  }\n  },\n  {\n  \"column\":
\"RetentionRate\",\n  \"properties\": {\n  \"dtype\":
\"number\",\n  \"std\": 21.015542555187874,\n  \"min\":
57.29032258064516,\n  \"max\": 97.16814159292035,\n
\"num_unique_values\": 3,\n  \"samples\": [\n
57.29032258064516,\n  88.73048200950441,\n
97.16814159292035\n  ],\n  \"semantic_type\": \"\",\n
\"description\": \"\"\n  }\n  }\n  ]\n}","type":"dataframe"}
```


```
plt.figure(figsize=(10, 6))
sns.barplot(x=contract_churn.index, y='RetentionRate',
data=contract_churn, palette='viridis', hue=contract_churn.index,
legend=False)
plt.title('Retention Rate by Contract Type')
plt.xlabel('Contract Type')
plt.ylabel('Retention Rate (%)')
plt.ylim(0, 100)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```



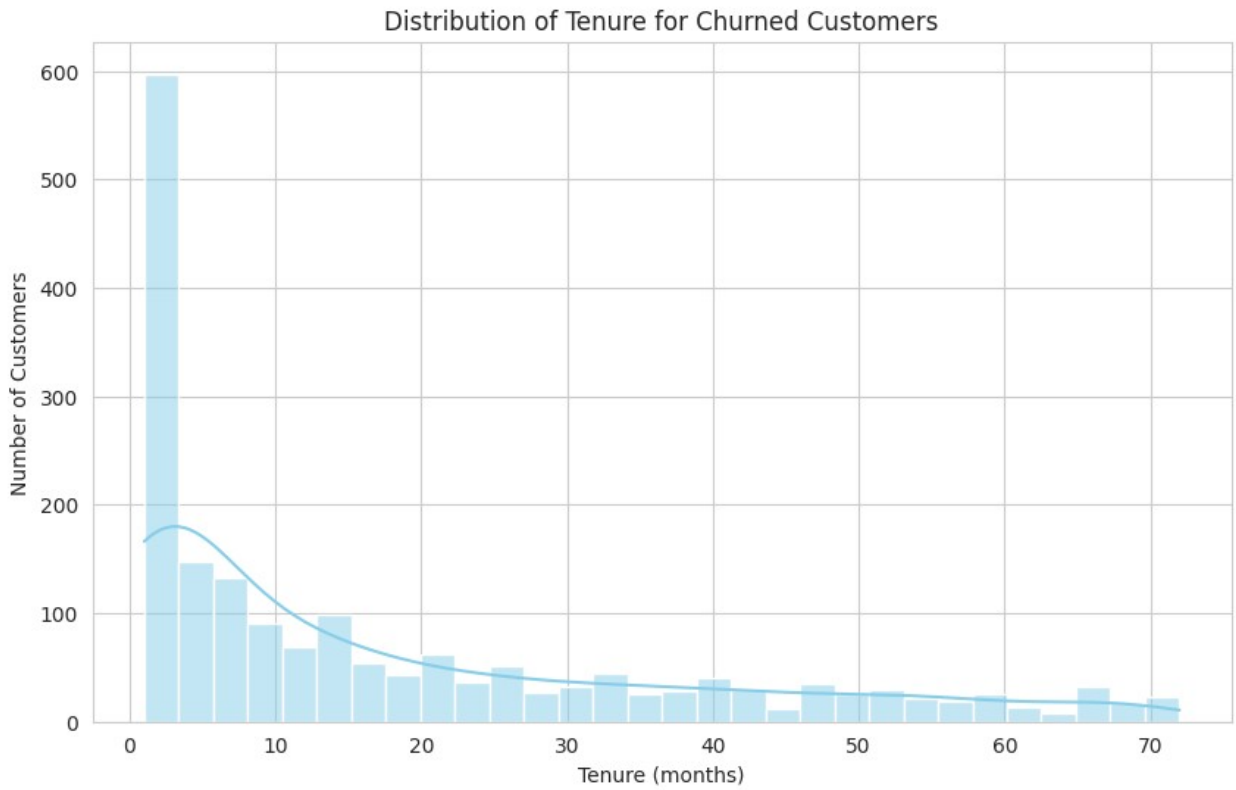

```
# Visualize churn rate by contract type
plt.figure(figsize=(10, 6))
sns.barplot(x=contract_churn.index, y='ChurnRate',
data=contract_churn, palette='coolwarm', hue=contract_churn.index,
legend=False)
plt.title('Churn Rate by Contract Type')
plt.xlabel('Contract Type')
plt.ylabel('Churn Rate (%)')
plt.ylim(0, 100)
plt.show()
```




```
# Filter for churned customers
churned_customers = df[df['Churn'] == 'Yes']

# Calculate the average tenure for churned customers
average_tenure_churned = churned_customers['tenure'].mean()

print(f"Average tenure for churned customers:
{average_tenure_churned:.2f} months")
```

```
Average tenure for churned customers: 17.98 months
```

```
import matplotlib.pyplot as plt
import seaborn as sns

# Set the aesthetic style of the plots
sns.set_style("whitegrid")

# Create a histogram to visualize the distribution of 'tenure' for
churned customers
plt.figure(figsize=(10, 6))
sns.histplot(churned_customers['tenure'], bins=30, kde=True,
color='skyblue')
plt.title('Distribution of Tenure for Churned Customers')
plt.xlabel('Tenure (months)')
plt.ylabel('Number of Customers')
plt.show()
```



Retention Strategy Proposal for High-Churn Risk Segments

Based on the analysis, several customer segments exhibit a higher propensity to churn. A targeted retention strategy should focus on these groups to maximize impact.

1. Customers on Month-to-Month Contracts

- **Insight:** This segment has the highest churn rate (42.71%), indicating low commitment.
- **Strategy:**
  - **Incentivize Longer-Term Contracts:** Offer discounts, exclusive features, or premium service upgrades for customers who switch to 1-year or 2-year contracts. Highlight the cost savings and benefits of commitment.
  - **Proactive Engagement:** Implement a communication strategy that educates these customers on the value of their current services and potential benefits of longer commitments before their contract renewal date.
  - **Personalized Offers:** Use data on their service usage to tailor contract upgrade offers that align with their needs.

2. Customers Using Electronic Check Payment Method

- **Insight:** This payment method is associated with a significantly higher churn rate (45.29%).
- **Strategy:**

  - **Promote Alternative Payment Methods:** Offer incentives (e.g., small discounts, bonus points) for customers to switch to more stable payment methods like credit card (automatic) or bank transfer (automatic).
  - **Improve Payment Experience:** Investigate if there are any friction points or issues associated with electronic check payments that contribute to dissatisfaction.

3. Senior Citizen Customers

- **Insight:** Senior citizens have a notably higher churn rate (41.68%).
- **Strategy:**
  - **Dedicated Support Channels:** Offer easily accessible and patient customer support tailored to the needs of senior citizens.
  - **Simplified Plans & Features:** Develop or highlight simpler, value-driven plans that meet their specific needs, avoiding overwhelming them with complex offerings.
  - **Digital Literacy Support:** Provide resources or workshops to help them navigate digital services, if complexity is a barrier.

4. Customers Without Dependents

- **Insight:** This group shows a higher churn rate (31.28%) compared to those with dependents.
- **Strategy:**
  - **Focus on Individual Benefits:** Tailor marketing and communication to highlight benefits that appeal to individuals (e.g., flexibility, mobile features, entertainment packages).
  - **Community Building:** Create exclusive communities or events that foster connection, as these customers might be seeking social engagement.

5. Low Engagement & Low CLV Customers

- **Insight:** These customers contribute less revenue and are highly susceptible to churn (Low Engagement: 44.49% churn; Low CLV: 39.25% churn).
- **Strategy:**
  - **Re-engagement Campaigns:** Identify triggers for low engagement (e.g., infrequent usage of key services) and launch targeted campaigns to re-engage them with service benefits or new offerings.
  - **Value Proposition Reinforcement:** Educate them on the full value of their current services to increase perceived value.
  - **Tiered Offerings:** For very low CLV customers, consider streamlined, cost-effective service tiers that still provide core value while managing acquisition costs.

6. Customers with Low Tenure (Early Churners)

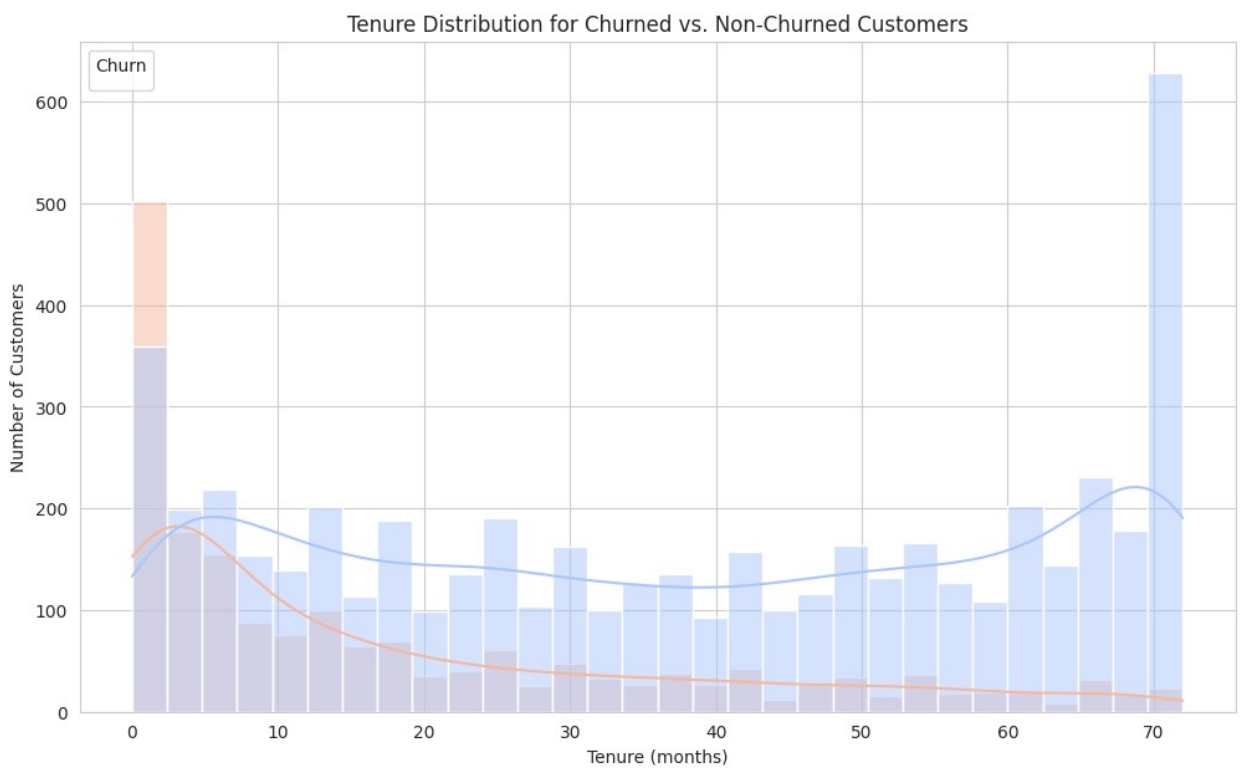
- **Insight:** The highest churn rates are observed in the 0-12 month tenure group (48.28%).
- **Strategy:**
  - **Enhanced Onboarding:** Implement a robust onboarding program to ensure new customers fully understand and utilize their services, addressing any early pain points.

- **Proactive Check-ins:** Conduct early check-ins (e.g., after 1, 3, and 6 months) to gather feedback, resolve issues, and ensure satisfaction.
- **Welcome Incentives:** Offer small, short-term incentives to encourage continued usage and build initial loyalty.

By systematically addressing the unique needs and churn drivers of these high-risk segments, the company can significantly improve its overall retention rates and safeguard valuable revenue.

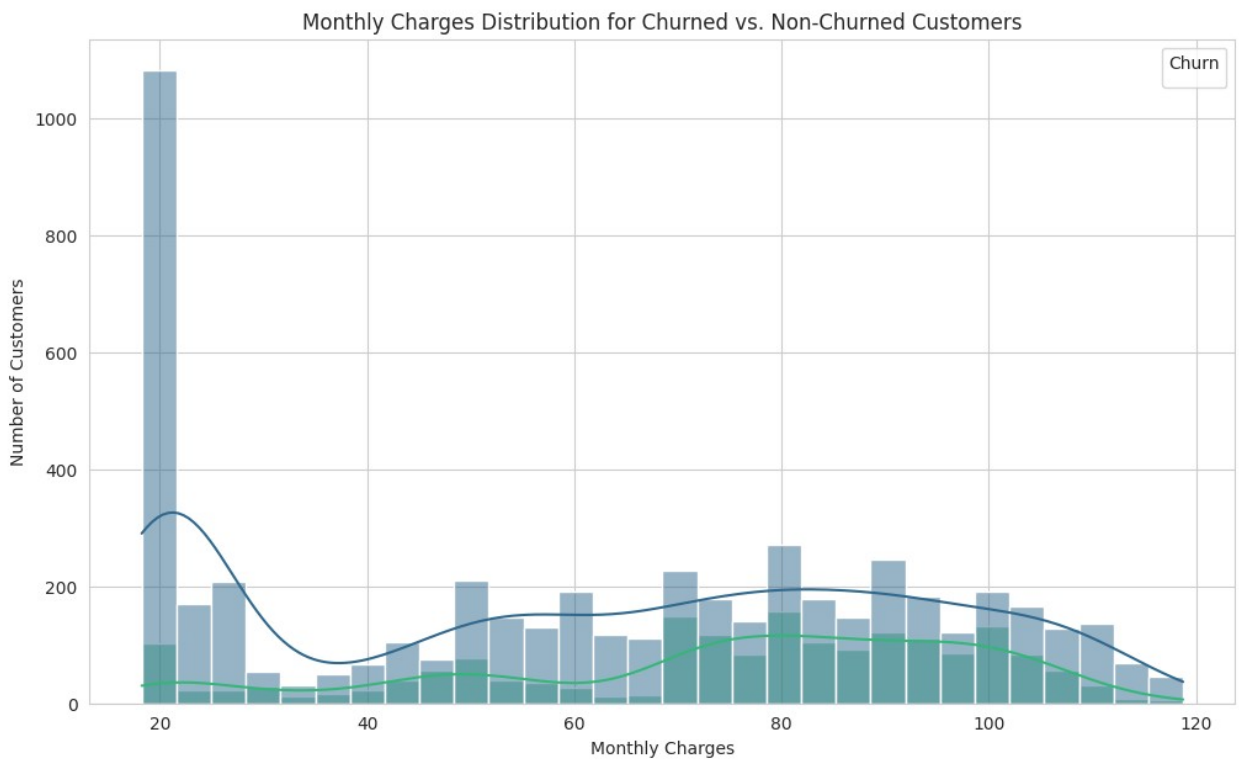
```
non_churned_customers = df[df['Churn'] == 'No']

plt.figure(figsize=(12, 7))
sns.histplot(data=df, x='tenure', hue='Churn', bins=30, kde=True,
palette='coolwarm')
plt.title('Tenure Distribution for Churned vs. Non-Churned Customers')
plt.xlabel('Tenure (months)')
plt.ylabel('Number of Customers')
plt.legend(title='Churn')
plt.show()
```

```
/tmp/ipykernel_9239/156501890.py:8: UserWarning: No artists with
labels found to put in legend.  Note that artists whose label start
with an underscore are ignored when legend() is called with no
argument.
  plt.legend(title='Churn')
```



```
plt.figure(figsize=(12, 7))
sns.histplot(data=df, x='MonthlyCharges', hue='Churn', bins=30,
kde=True, palette='viridis')
plt.title('Monthly Charges Distribution for Churned vs. Non-Churned
Customers')
plt.xlabel('Monthly Charges')
plt.ylabel('Number of Customers')
plt.legend(title='Churn')
plt.show()
```

```
/tmp/ipykernel_9239/804677696.py:6: UserWarning: No artists with
labels found to put in legend.  Note that artists whose label start
with an underscore are ignored when legend() is called with no
argument.
  plt.legend(title='Churn')
```

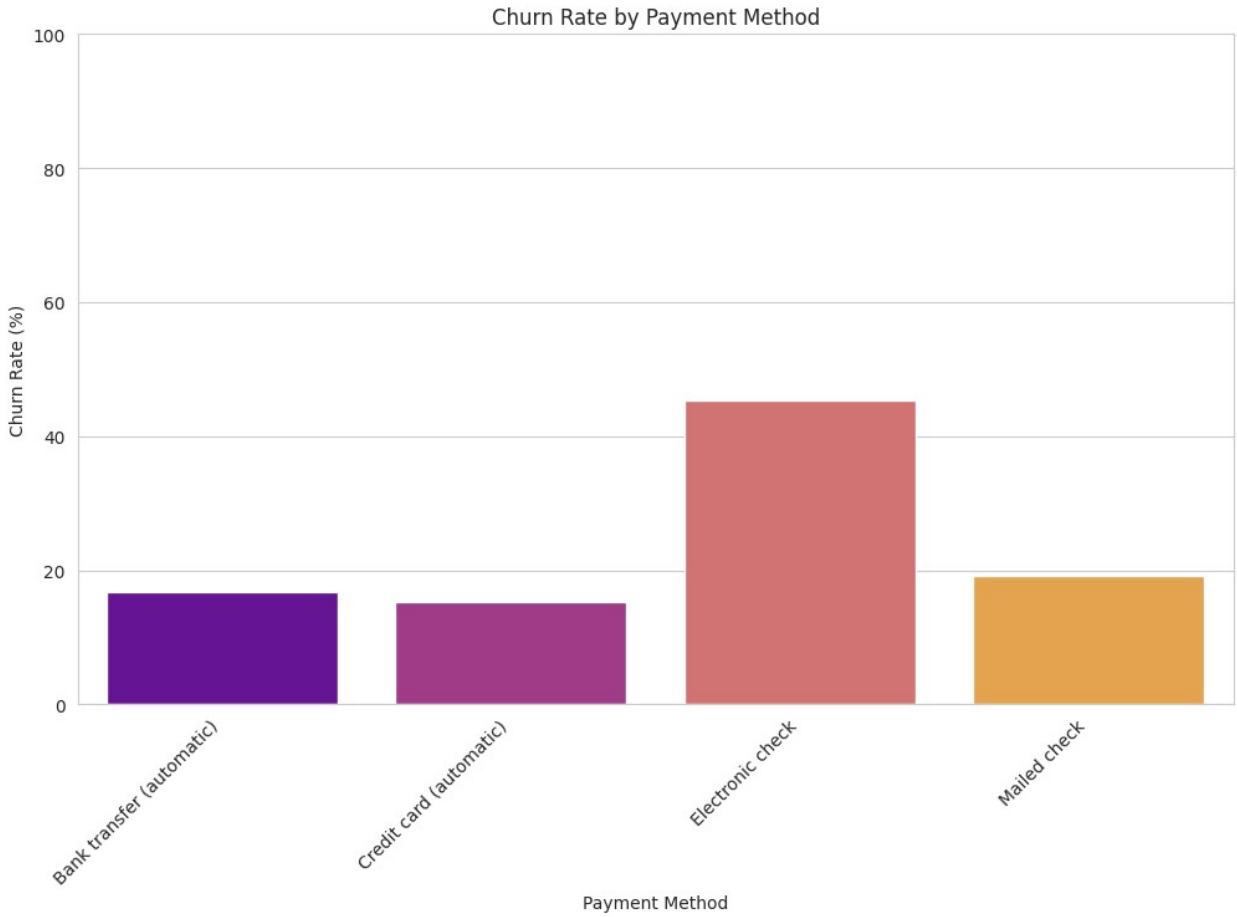



```
# Calculate churn rate by payment method
payment_churn = df.groupby(['PaymentMethod',
'Churn']).size().unstack(fill_value=0)
payment_churn['Total'] = payment_churn['Yes'] + payment_churn['No']
payment_churn['ChurnRate'] = (payment_churn['Yes'] /
payment_churn['Total']) * 100

print("Churn Rate by Payment Method:")
display(payment_churn)
```

```
Churn Rate by Payment Method:
```


```
{"summary":"{\n  \"name\": \"payment_churn\",\n  \"rows\": 4,\n  \"fields\": [\n  {\n  \"column\": \"PaymentMethod\",\n  \"properties\": {\n  \"dtype\": \"string\",\n  \"num_unique_values\": 4,\n  \"samples\": [\n  \"Credit card (automatic)\",\n  \"Mailed check\",\n  \"Bank transfer (automatic)\"\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"No\",\n  \"properties\": {\n  \"dtype\": \"number\",\n  \"std\": 7,\n  \"min\": 1286,\n  \"max\": 1304,\n  \"num_unique_values\": 4,\n  \"samples\": [\n  1290,\n  1304,\n  1286\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"Yes\",\n  \"properties\": {\n  \"dtype\": \"number\",\n  \"std\": 403,\n  \"min\": 232,\n  \"max\": 1071,\n  \"num_unique_values\": 4,\n  \"samples\": [\n  232,\n  308,\n  258\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"Total\",\n  \"properties\": {\n  \"dtype\": \"number\",\n  \"std\": 404,\n  \"min\": 1522,\n  \"max\": 2365,\n  \"num_unique_values\": 4,\n  \"samples\": [\n  1522,\n  1612,\n  1544\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"ChurnRate\",\n  \"properties\": {\n  \"dtype\": \"number\",\n  \"std\": 14.22220141793858,\n  \"min\": 15.243101182654403,\n  \"max\": 45.28541226215645,\n  \"num_unique_values\": 4,\n  \"samples\": [\n  15.243101182654403,\n  19.106699751861044,\n  16.709844559585495\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  }\n  ]\n}","type":"dataframe","variable_name":"payment_churn"}
```


```
# Visualize churn rate by payment method
plt.figure(figsize=(12, 7))
sns.barplot(x=payment_churn.index, y='ChurnRate', data=payment_churn,
palette='plasma', hue=payment_churn.index, legend=False)
plt.title('Churn Rate by Payment Method')
plt.xlabel('Payment Method')
plt.ylabel('Churn Rate (%)')
plt.ylim(0, 100)
plt.xticks(rotation=45, ha='right')
plt.show()
```



Retention Rate by Payment Method

Following the churn rate analysis, let's visualize the retention rate for each payment method. Retention rate is calculated as (100% - Churn Rate).

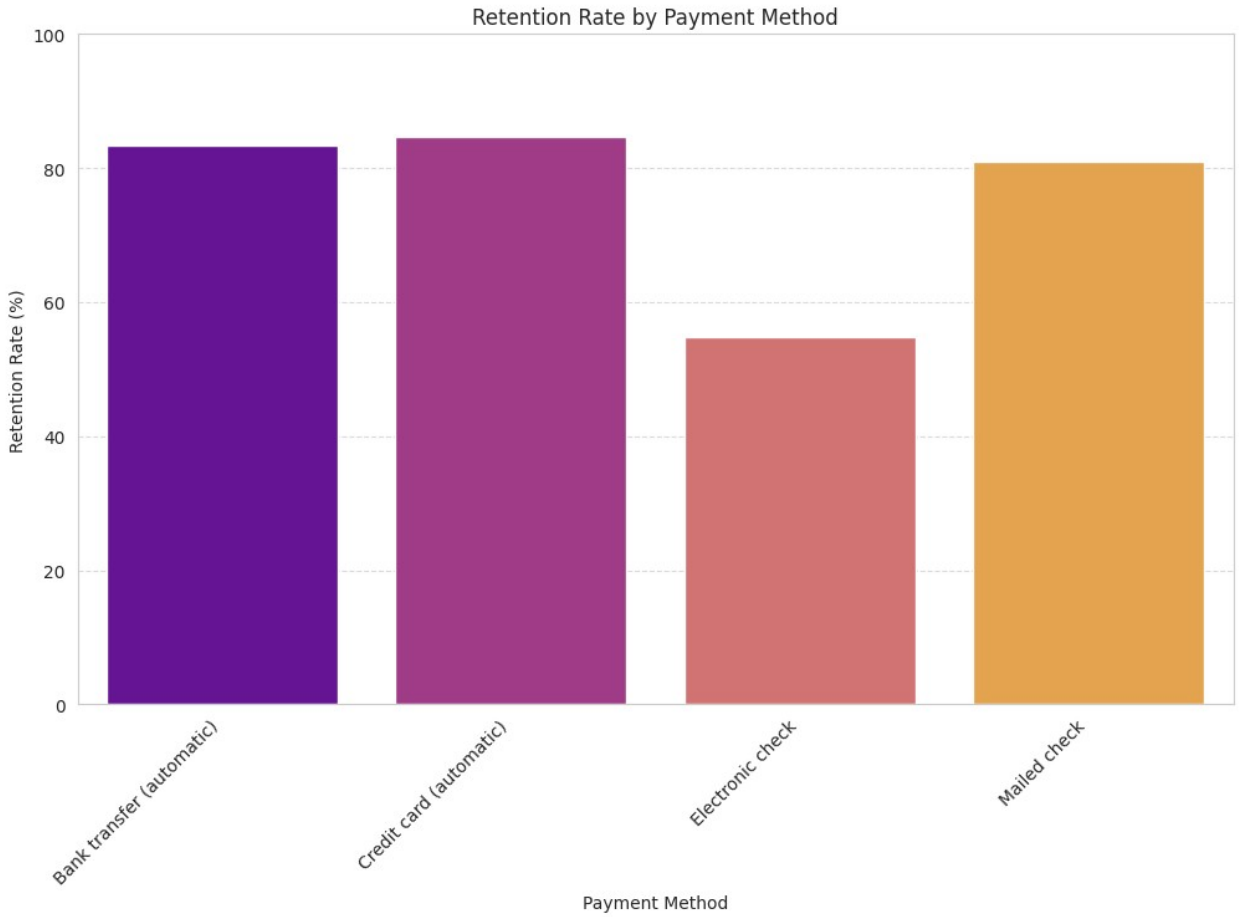
```
# Calculate Retention Rate by Payment Method
payment_churn['RetentionRate'] = 100 - payment_churn['ChurnRate']

print("Retention Rate by Payment Method:")
display(payment_churn[['Total', 'RetentionRate']])
```

```
Retention Rate by Payment Method:
```


```
{"summary":"{\n  \"name\": \"display(payment_churn[['Total',
'RetentionRate']])\",\n  \"rows\": 4,\n  \"fields\": [\n  {\n
\"column\": \"PaymentMethod\",\n      \"properties\": {\n
\"dtype\": \"string\",\n        \"num_unique_values\": 4,\n
\"samples\": [\n          \"Credit card (automatic)\",\n
\"Mailed check\",\n          \"Bank transfer (automatic)\"\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Total\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
```

\"std\": 404,\n   \"min\": 1522,\n   \"max\": 2365,\n   \"num_unique_values\": 4,\n   \"samples\": [\n   1522,\n   1612,\n   1544\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"RetentionRate\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 14.222201417938582,\n   \"min\": 54.71458773784355,\n   \"max\": 84.7568988173456,\n   \"num_unique_values\": 4,\n   \"samples\": [\n   84.7568988173456,\n   80.89330024813896,\n   83.29015544041451\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe"}

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12, 7))
sns.barplot(x=payment_churn.index, y='RetentionRate',
data=payment_churn, palette='plasma', hue=payment_churn.index,
legend=False)
plt.title('Retention Rate by Payment Method')
plt.xlabel('Payment Method')
plt.ylabel('Retention Rate (%)')
plt.ylim(0, 100)
plt.xticks(rotation=45, ha='right')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```



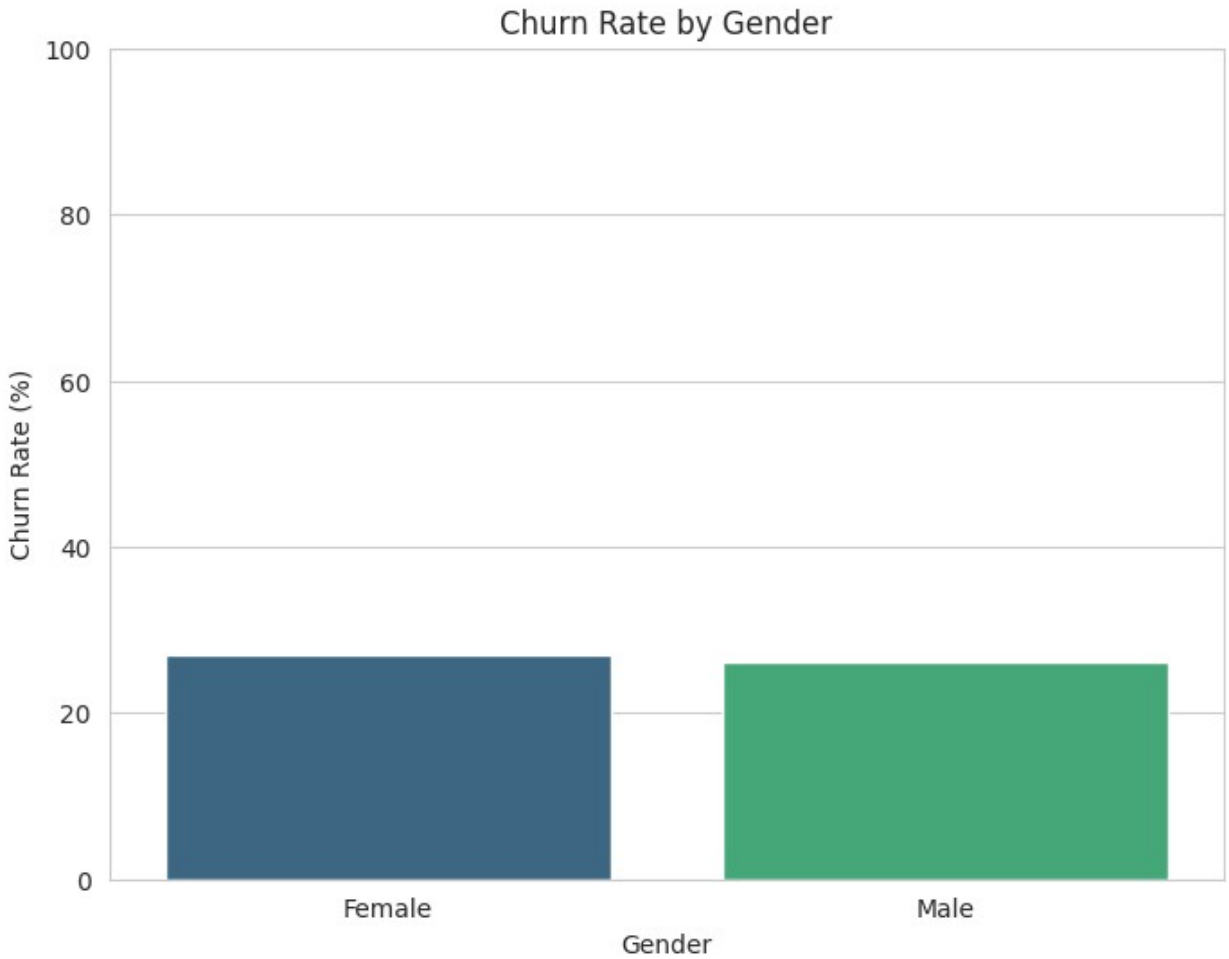

```
# Calculate churn rate by gender
gender_churn = df.groupby(['gender',
'Churn']).size().unstack(fill_value=0)
gender_churn['Total'] = gender_churn['Yes'] + gender_churn['No']
gender_churn['ChurnRate'] = (gender_churn['Yes'] /
gender_churn['Total']) * 100

print("Churn Rate by Gender:")
display(gender_churn)
```

```
Churn Rate by Gender:
```


```
{"summary":"{\n  \"name\": \"gender_churn\",\n  \"rows\": 2,\n
\"fields\": [\n  {\n  \"column\": \"gender\",\n
\"properties\": {\n  \"dtype\": \"string\",\n
\"num_unique_values\": 2,\n  \"samples\": [\n
\"Male\",\n  \"Female\"\n  ],\n
\"semantic_type\": \"\",\n  \"description\": \"\"\n  }\
n  },\n  {\n  \"column\": \"No\",\n  \"properties\": {\n
\"dtype\": \"number\",\n  \"std\": 53,\n  \"min\": 2549,\n
\"max\": 2625,\n  \"num_unique_values\": 2,\n
\"samples\": [\n  2625,\n  2549\n  ],\n
```

\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Yes\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 6,\n   \"min\": 930,\n   \"max\": 939,\n   \"num_unique_values\": 2,\n   \"samples\": [\n   930,\n   939\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Total\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 47,\n   \"min\": 3488,\n   \"max\": 3555,\n   \"num_unique_values\": 2,\n   \"samples\": [\n   3555,\n   3488\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"ChurnRate\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 0.5377787535951827,\n   \"min\": 26.160337552742618,\n   \"max\": 26.920871559633024,\n   \"num_unique_values\": 2,\n   \"samples\": [\n   26.160337552742618,\n   26.920871559633024\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe","variable_name":"gender_churn"}

```
# Visualize churn rate by gender
plt.figure(figsize=(8, 6))
sns.barplot(x=gender_churn.index, y='ChurnRate', data=gender_churn,
palette='viridis', hue=gender_churn.index, legend=False)
plt.title('Churn Rate by Gender')
plt.xlabel('Gender')
plt.ylabel('Churn Rate (%)')
plt.ylim(0, 100)
plt.show()
```



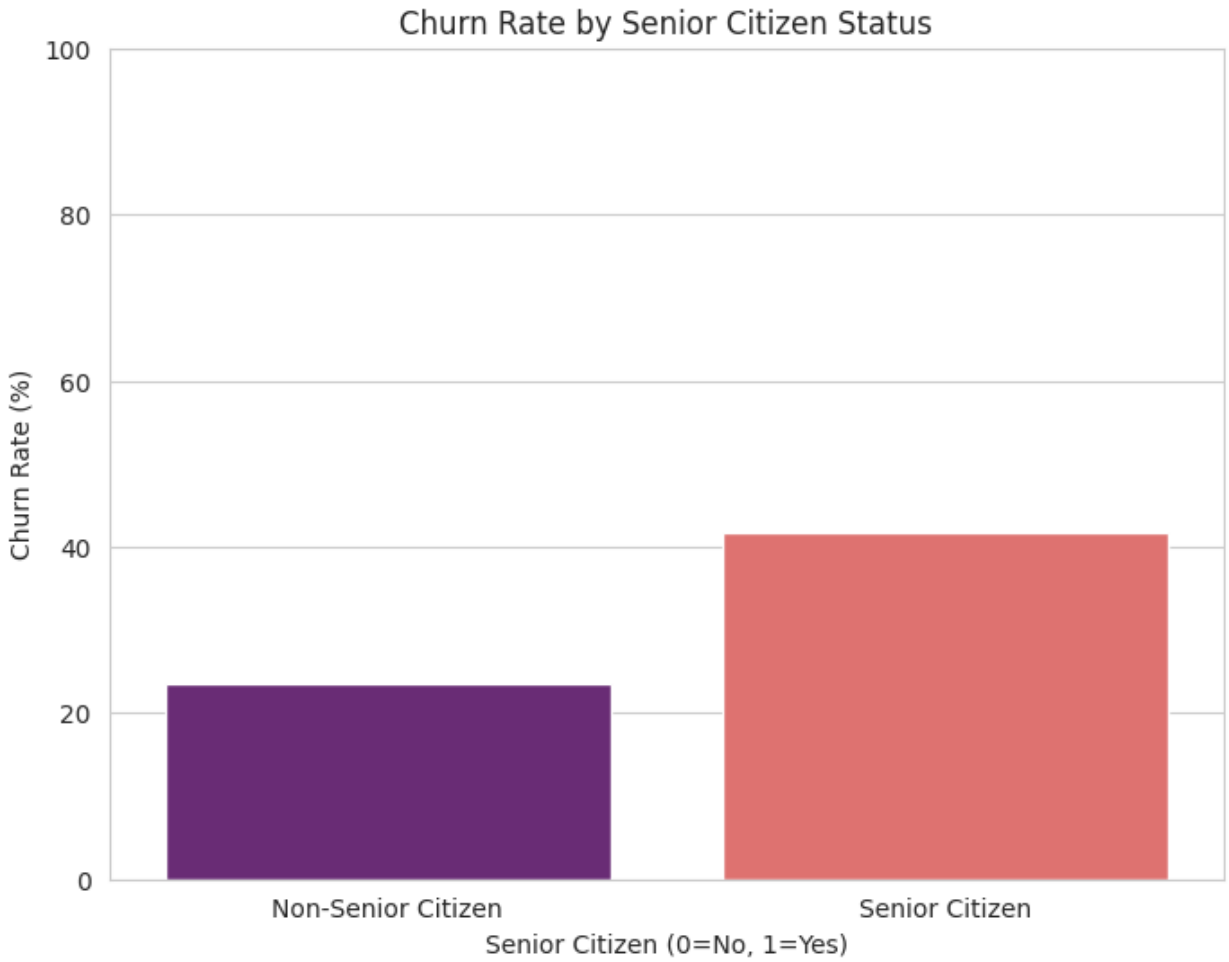

```
# Calculate churn rate by SeniorCitizen status
senior_citizen_churn = df.groupby(['SeniorCitizen',
'Churn']).size().unstack(fill_value=0)
senior_citizen_churn['Total'] = senior_citizen_churn['Yes'] +
senior_citizen_churn['No']
senior_citizen_churn['ChurnRate'] = (senior_citizen_churn['Yes'] /
senior_citizen_churn['Total']) * 100

print("Churn Rate by Senior Citizen Status:")
display(senior_citizen_churn)
```

```
Churn Rate by Senior Citizen Status:
```

{"summary":"{\n \"name\": \"senior_citizen_churn\",\n \"rows\": 2,\n \"fields\": [\n {\n \"column\": \"SeniorCitizen\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0,\n \"min\": 0,\n \"max\": 1,\n \"num_unique_values\": 2,\n \"samples\": [\n 1,\n 0\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\":

\"No\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 2716,\n   \"min\": 666,\n   \"max\": 4508,\n
\"num_unique_values\": 2,\n   \"samples\": [\n   666,\n
4508\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"Yes\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 648,\n   \"min\": 476,\n   \"max\": 1393,\n
\"num_unique_values\": 2,\n   \"samples\": [\n   476,\n
1393\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"Total\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 3365,\n   \"min\": 1142,\n   \"max\": 5901,\n
\"num_unique_values\": 2,\n   \"samples\": [\n   1142,\n
5901\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"ChurnRate\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 12.781020477100085,\n   \"min\":
23.606168446026096,\n   \"max\": 41.68126094570928,\n
\"num_unique_values\": 2,\n   \"samples\": [\n
41.68126094570928,\n   23.606168446026096\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   }\n]\
n}","type":"dataframe","variable_name":"senior_citizen_churn"}

```
# Visualize churn rate by SeniorCitizen status
plt.figure(figsize=(8, 6))
sns.barplot(x=senior_citizen_churn.index, y='ChurnRate',
data=senior_citizen_churn, palette='magma',
hue=senior_citizen_churn.index, legend=False)
plt.title('Churn Rate by Senior Citizen Status')
plt.xlabel('Senior Citizen (0=No, 1=Yes)')
plt.ylabel('Churn Rate (%)')
plt.ylim(0, 100)
plt.xticks(ticks=[0, 1], labels=['Non-Senior Citizen', 'Senior
Citizen'])
plt.show()
```



Churn Rate by Dependents Status

Let's analyze the churn rate for customers based on whether they have dependents or not. This can provide insights into how family status might influence churn behavior.

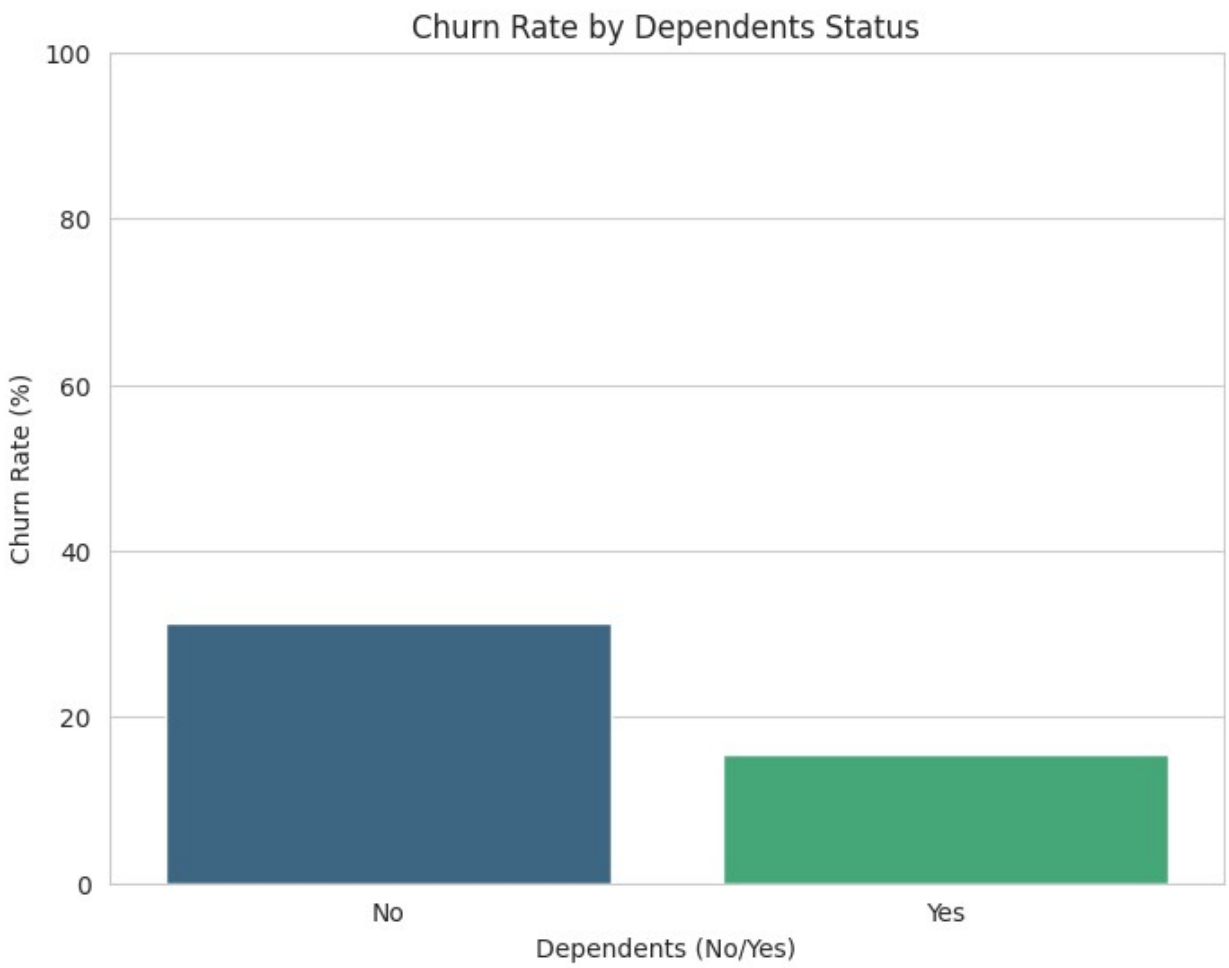
```
# Calculate churn rate by Dependents status
dependents_churn = df.groupby(['Dependents',
'Churn']).size().unstack(fill_value=0)
dependents_churn['Total'] = dependents_churn['Yes'] +
dependents_churn['No']
dependents_churn['ChurnRate'] = (dependents_churn['Yes'] /
dependents_churn['Total']) * 100

print("Churn Rate by Dependents Status:")
display(dependents_churn)
```

```
Churn Rate by Dependents Status:
```


```
{"summary":"{\n  \"name\": \"dependents_churn\",\n  \"rows\": 2,\n
\"fields\": [\n  {\n    \"column\": \"Dependents\",\n
```

\"properties\": {\n   \"dtype\": \"string\",\n
\"num_unique_values\": 2,\n   \"samples\": [\n   \"Yes\",\
n   \"No\"\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"No\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 1135,\n   \"min\": 1784,\n   \"max\": 3390,\n
\"num_unique_values\": 2,\n   \"samples\": [\n   1784,\n
3390\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"Yes\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 860,\n   \"min\": 326,\n   \"max\": 1543,\n
\"num_unique_values\": 2,\n   \"samples\": [\n   326,\n
1543\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"Total\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 1996,\n   \"min\": 2110,\n   \"max\": 4933,\n
\"num_unique_values\": 2,\n   \"samples\": [\n   2110,\n
4933\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"ChurnRate\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 11.192725014656896,\n   \"min\":
15.450236966824646,\n   \"max\": 31.279140482465028,\n
\"num_unique_values\": 2,\n   \"samples\": [\n
15.450236966824646,\n   31.279140482465028\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   }\n]\n}","type":"dataframe","variable_name":"dependents_churn"}

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8, 6))
sns.barplot(x=dependents_churn.index, y='ChurnRate',
data=dependents_churn, palette='viridis', hue=dependents_churn.index,
legend=False)
plt.title('Churn Rate by Dependents Status')
plt.xlabel('Dependents (No/Yes)')
plt.ylabel('Churn Rate (%)')
plt.ylim(0, 100)
plt.show()
```



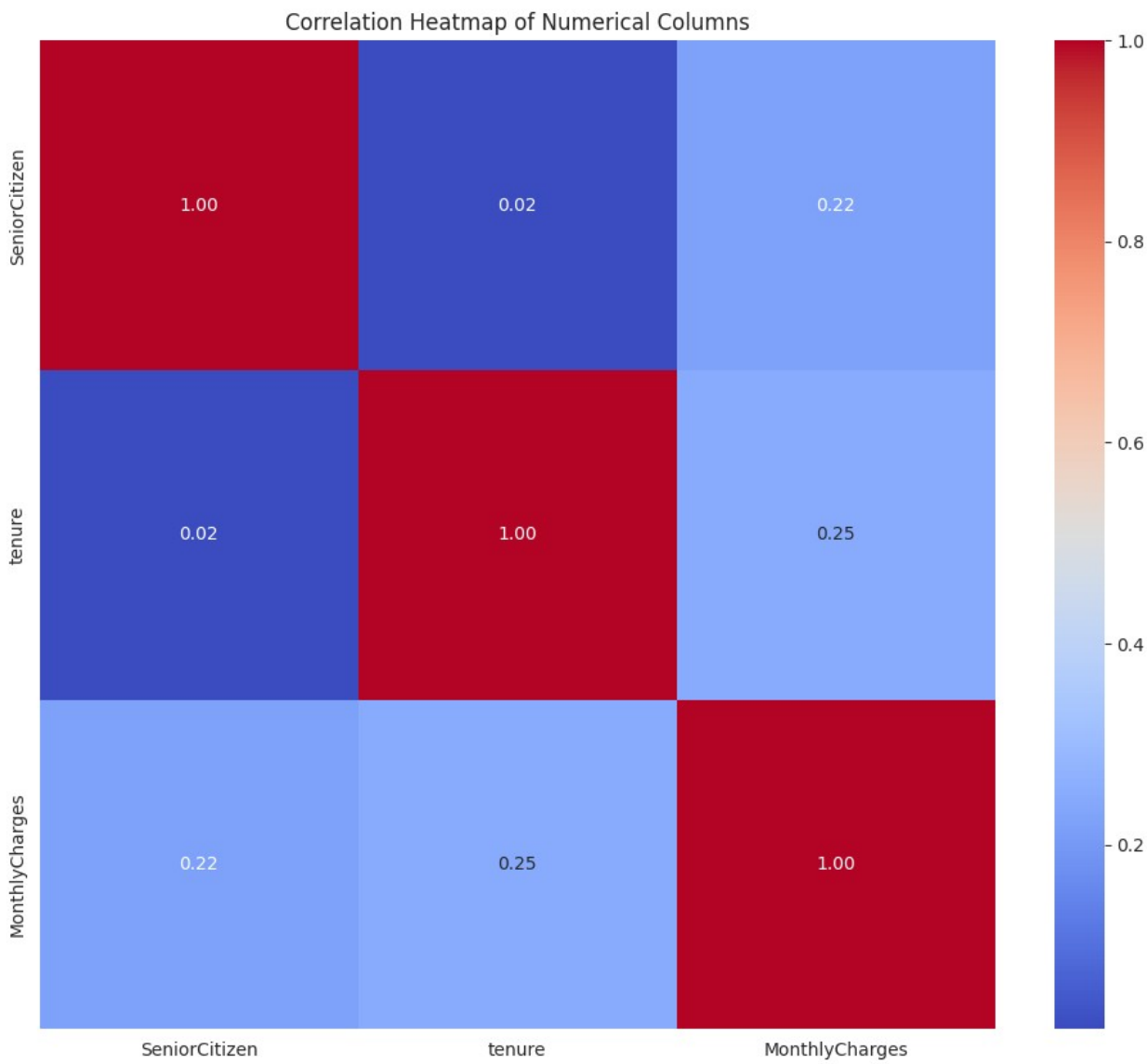
Overall Churn and Retention Rates

Let's calculate the overall churn rate and then derive the overall retention rate for the entire customer base.

```
# Calculate overall churn rate
churn_count = df['Churn'].value_counts()['Yes']
total_customers = len(df)
overall_churn_rate = (churn_count / total_customers) * 100

# Calculate overall retention rate
overall_retention_rate = 100 - overall_churn_rate

print(f"Overall Churn Rate: {overall_churn_rate:.2f}%")
print(f"Overall Retention Rate: {overall_retention_rate:.2f}%")
```

```
Overall Churn Rate: 26.54%
Overall Retention Rate: 73.46%
```

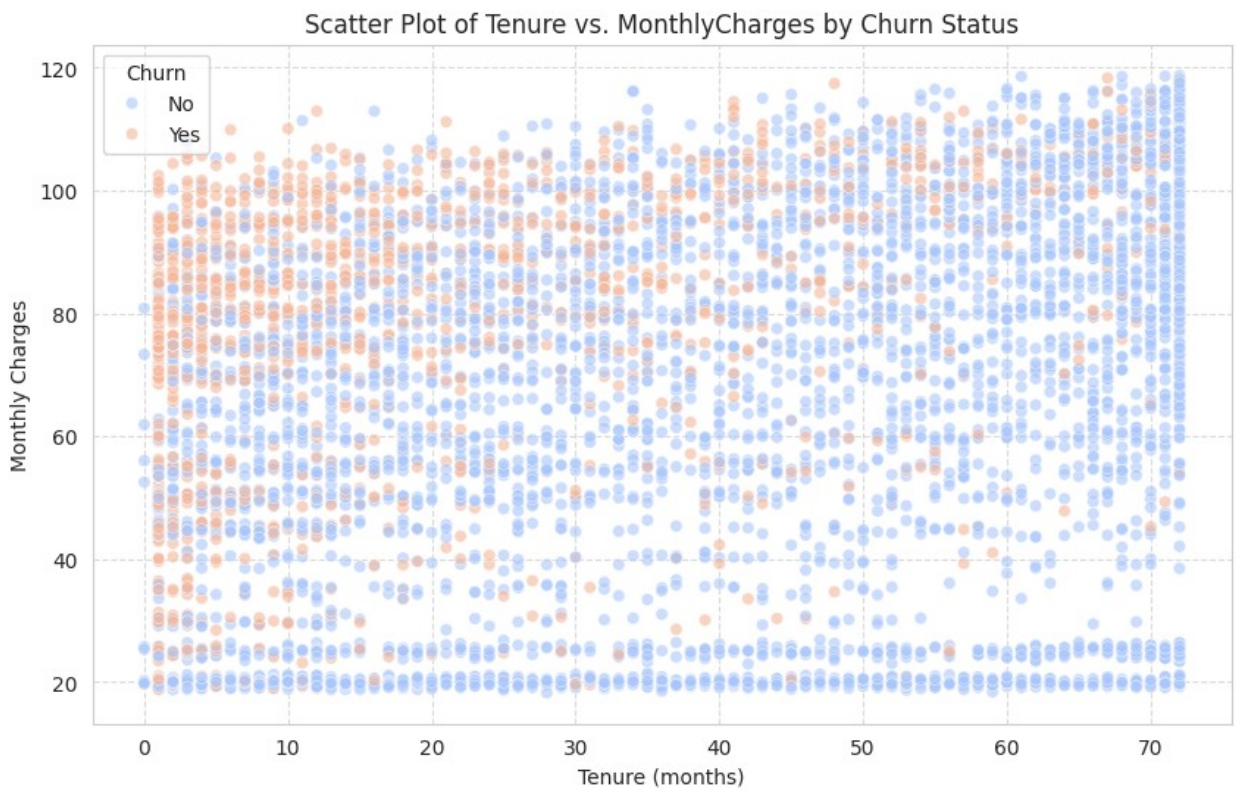
```
numerical_cols = df.select_dtypes(include=['number']).columns
correlation_matrix = df[numerical_cols].corr()

plt.figure(figsize=(12, 10))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm',
fmt='.2f')
plt.title('Correlation Heatmap of Numerical Columns')
plt.show()
```




```
print(f"Correlation between Tenure and MonthlyCharges:
{correlation_matrix.loc['tenure', 'MonthlyCharges']:.2f}")
```

```
Correlation between Tenure and MonthlyCharges: 0.25
```

```
plt.figure(figsize=(10, 6))
sns.scatterplot(data=df, x='tenure', y='MonthlyCharges', hue='Churn',
palette='coolwarm', alpha=0.6)
plt.title('Scatter Plot of Tenure vs. MonthlyCharges by Churn Status')
plt.xlabel('Tenure (months)')
plt.ylabel('Monthly Charges')
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```



Analysis of Correlation between Tenure and Monthly Charges

The correlation coefficient between 'tenure' and 'MonthlyCharges' is **0.25**, indicating a weak positive correlation. This means that as a customer's tenure increases, their monthly charges tend to slightly increase, but the relationship is not very strong.

The scatter plot further illustrates this:

- There isn't a very tight linear relationship, confirming the weak correlation.
- We can observe clusters of customers with low tenure and low monthly charges, as well as customers with high tenure and high monthly charges.
- Interestingly, the churned customers (red points) appear to be more concentrated in the areas of lower tenure and higher monthly charges, especially in the early stages of their contracts. This aligns with earlier findings where churned customers had shorter tenures and higher monthly charges.

Average Monthly Charges by Tenure Intervals

To understand how monthly charges relate to customer loyalty, we will group customers into tenure intervals and calculate the average monthly charge for each group.

```
# Define tenure intervals
bins = [0, 12, 24, 36, 48, 60, 72, df['tenure'].max() + 1] # +1 to
include the max tenure
labels = ['0-12', '13-24', '25-36', '37-48', '49-60', '61-72', '72+']
df['TenureGroup'] = pd.cut(df['tenure'], bins=bins, labels=labels,
right=False)

# Calculate average monthly charges by tenure group
average_monthly_charges_by_tenure = df.groupby('TenureGroup',
observed=False)['MonthlyCharges'].mean().reset_index()

print("Average Monthly Charges by Tenure Group:")
display(average_monthly_charges_by_tenure)
```

```
Average Monthly Charges by Tenure Group:
```


```
{"summary":"{\n  \"name\": \"average_monthly_charges_by_tenure\",\n
\"rows\": 7,\n  \"fields\": [\n  {\n  \"column\":
\"TenureGroup\",\n  \"properties\": {\n  \"dtype\":
\"category\",\n  \"num_unique_values\": 7,\n  \"samples\":
[\n  \"0-12\",\n  \"13-24\",\n  \"61-72\"\n
],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n
}\n  },\n  {\n  \"column\": \"MonthlyCharges\",\n
\"properties\": {\n  \"dtype\": \"number\",\n  \"std\":
8.237771874583382,\n  \"min\": 56.05601739971,\n  \"max\":
80.69585635359117,\n  \"num_unique_values\": 7,\n
\"samples\": [\n  56.05601739971,\n
60.85329512893983,\n  74.29661016949153\n  ],\n
\"semantic_type\": \"\",\n  \"description\": \"\"\n  }\
n  }\n  ]\
n}","type":"dataframe","variable_name":"average_monthly_charges_by_ten
ure"}
```


Calculating Engagement Score

To derive an 'engagement score', we'll combine three key aspects:

1. **Tenure**: Directly reflects how long a customer has been with the service.
2. **Contract Type**: Longer contracts generally imply higher commitment/engagement.
3. **Total Services**: The number of additional services a customer subscribes to, indicating deeper integration with the provider.

We'll calculate 'total services' by counting the number of 'Yes' responses in service-related columns and map contract types to numerical values. Finally, we'll combine these into a composite engagement score.

```python
# 1. Calculate 'total_services'

service_columns = [
    'PhoneService',
    'MultipleLines',
    'InternetService',
    'OnlineSecurity',
    'OnlineBackup',
    'DeviceProtection',
    'TechSupport',
    'StreamingTV',
    'StreamingMovies'
]

# Initialize 'total_services' to 0
df['total_services'] = 0

# Count 'Yes' for most services
for col in ['PhoneService', 'MultipleLines', 'OnlineSecurity',
'OnlineBackup',
            'DeviceProtection', 'TechSupport', 'StreamingTV',
'StreamingMovies']:
    df['total_services'] += (df[col] == 'Yes').astype(int)

# Special handling for InternetService: 'DSL' or 'Fiber optic' means
they have a service
df['total_services'] += df['InternetService'].apply(lambda x: 1 if x
in ['DSL', 'Fiber optic'] else 0)

print("Distribution of Total Services:")
display(df['total_services'].value_counts().sort_index())
```

```
Distribution of Total Services:

total_services
1    1264
2     859
3     846
4     965
5     922
6     908
7     676
8     395
9     208
Name: count, dtype: int64
```

```python
# 2. Map 'Contract' type to a numerical score
contract_mapping = {
    'Month-to-month': 1,
    'One year': 2,
```

```
    'Two year': 3
}
df['contract_score'] = df['Contract'].map(contract_mapping)

print("Distribution of Contract Score:")
display(df['contract_score'].value_counts().sort_index())
```

```
Distribution of Contract Score:

contract_score
1    3875
2    1473
3    1695
Name: count, dtype: int64
```

```
# 3. Calculate the 'engagement_score'
# A simple weighted sum: tenure + (contract_score * weight) +
(total_services * weight)
# We can adjust weights based on perceived importance. Let's start
with equal weighting for simplicity.
# We'll normalize tenure and total_services to bring them to a similar
scale before summing,
# so that one factor doesn't overwhelmingly dominate the score due to
its raw scale.

# Min-Max Scaling for 'tenure' and 'total_services'
df['tenure_scaled'] = (df['tenure'] - df['tenure'].min()) /
(df['tenure'].max() - df['tenure'].min())
df['total_services_scaled'] = (df['total_services'] -
df['total_services'].min()) / (df['total_services'].max() -
df['total_services'].min())

# For contract_score, it's already on a small integer scale (1, 2, 3).
We can use it directly or scale it too.
# Let's scale it to a similar range as the others (0-1), then apply
weights.
max_contract_score = df['contract_score'].max()
df['contract_score_scaled'] = df['contract_score'] /
max_contract_score

# Calculate engagement score
# Assigning arbitrary weights for demonstration; these could be tuned
or determined by business logic
weight_tenure = 0.4
weight_contract = 0.3
weight_services = 0.3

df['engagement_score'] = (
    df['tenure_scaled'] * weight_tenure +
    df['contract_score_scaled'] * weight_contract +
```

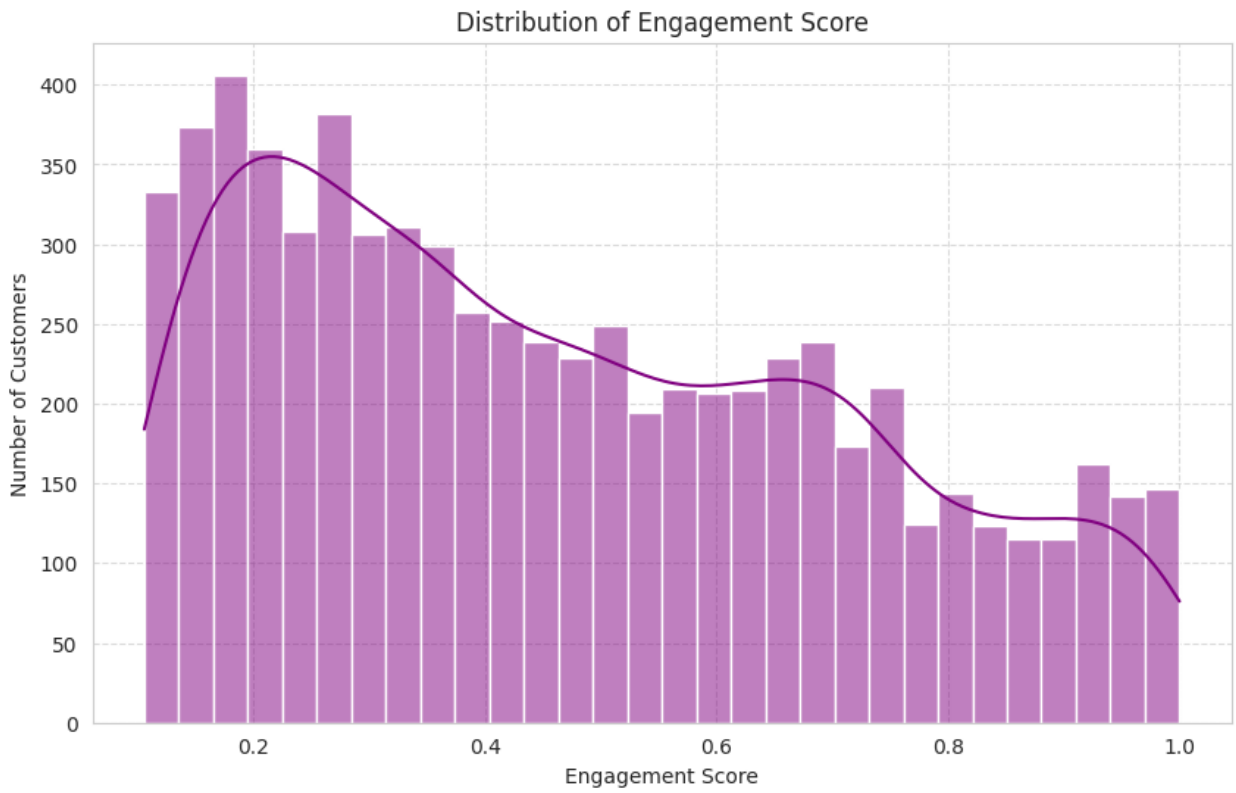
```
    df['total_services_scaled'] * weight_services
)

print("Descriptive Statistics for Engagement Score:")
display(df['engagement_score'].describe())

# Display the top 10 customers by engagement score
print("Top 10 Customers by Engagement Score:")
display(df[['customerID', 'tenure', 'Contract', 'total_services',
'engagement_score']].sort_values(by='engagement_score',
ascending=False).head(10))
```

```
Descriptive Statistics for Engagement Score:

count    7043.000000
mean        0.466871
std         0.250155
min         0.105556
25%         0.252083
50%         0.427778
75%         0.662500
max         1.000000
Name: engagement_score, dtype: float64

Top 10 Customers by Engagement Score:
```


{"summary":"{\n \"name\": \"display(df[['customerID', 'tenure', 'Contract', 'total_services', 'engagement_score']]\",\n \"rows\": 10,\n \"fields\": [\n {\n \"column\": \"customerID\",\n \"properties\": {\n \"dtype\": \"string\",\n \"num_unique_values\": 10,\n \"samples\": [\n \"5135-GRQJV\",\n \"3569-JFODW\",\n \"1779-PWPMG\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"tenure\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0,\n \"min\": 72,\n \"max\": 72,\n \"num_unique_values\": 1,\n \"samples\": [\n 72\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Contract\",\n \"properties\": {\n \"dtype\": \"category\",\n \"num_unique_values\": 1,\n \"samples\": [\n \"Two year\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"total_services\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0,\n \"min\": 9,\n \"max\": 9,\n \"num_unique_values\": 1,\n \"samples\": [\n 9\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"engagement_score\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0.0,\n \"min\": 1.0,\n \"max\": 1.0,\n

\"num_unique_values\": 1,\n   \"samples\": [\n   1.0\n],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n }\n   }\n]\n}","type":"dataframe"}

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 6))
sns.histplot(df['engagement_score'], bins=30, kde=True,
color='purple')
plt.title('Distribution of Engagement Score')
plt.xlabel('Engagement Score')
plt.ylabel('Number of Customers')
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```



Analysis of Engagement Score

The engagement score provides a composite view of customer loyalty and service usage. By combining `tenure`, `Contract` type, and `total_services`, we've created a metric that can potentially highlight highly engaged customers versus those who are less engaged.

- The distribution plot above helps visualize the spread of engagement scores across the customer base.

- Further analysis could involve correlating this `engagement_score` with churn, or using it as a feature in predictive models, or segmenting customers based on their engagement levels.

Churn Rate by Tenure-Based Cohort

Let's analyze the churn rate for different tenure groups to understand how customer loyalty (represented by tenure) impacts their likelihood of churning. We will use the `TenureGroup` column previously created.

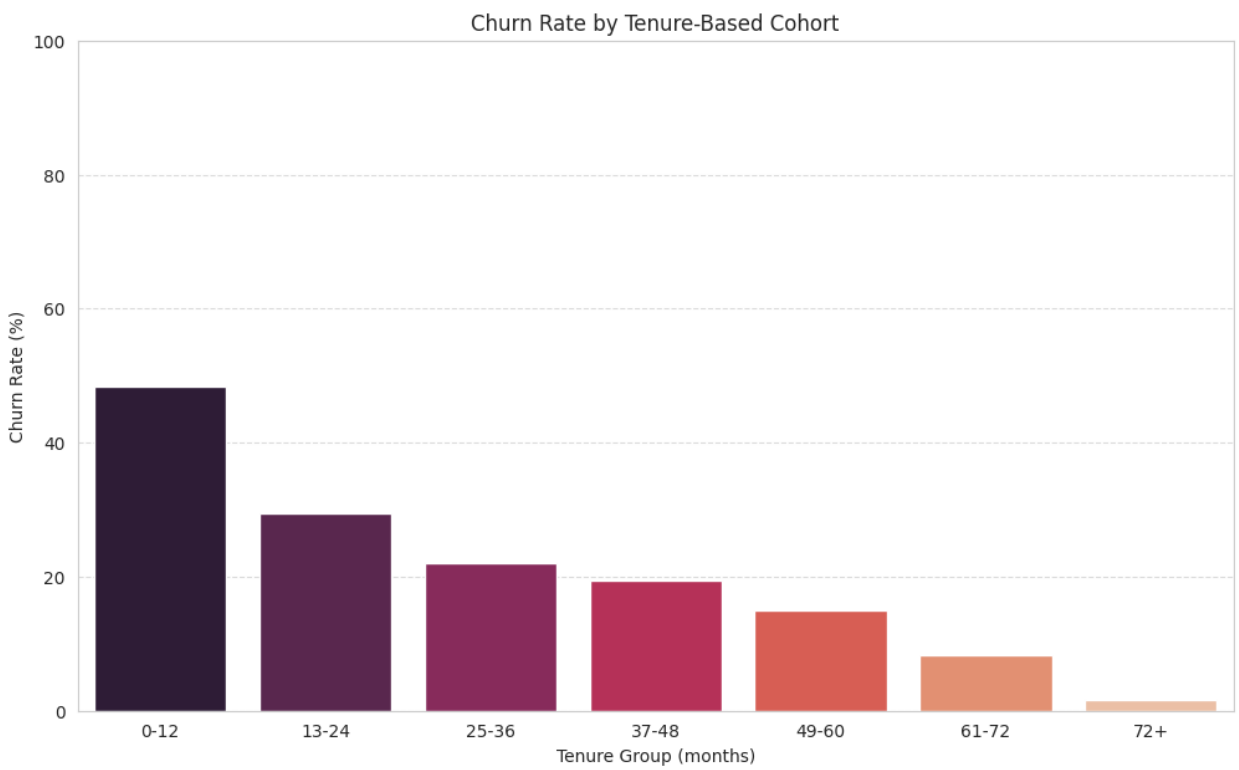
```
# Calculate churn rate by TenureGroup
tenure_cohort_churn = df.groupby(['TenureGroup', 'Churn'],
observed=False).size().unstack(fill_value=0)
tenure_cohort_churn['Total'] = tenure_cohort_churn['Yes'] +
tenure_cohort_churn['No']
tenure_cohort_churn['ChurnRate'] = (tenure_cohort_churn['Yes'] /
tenure_cohort_churn['Total']) * 100

print("Churn Rate by Tenure-Based Cohort:")
display(tenure_cohort_churn)
```

```
Churn Rate by Tenure-Based Cohort:
```


```
{"summary":"{\n  \"name\": \"tenure_cohort_churn\",\n  \"rows\": 7,\n
\"fields\": [\n  {\n  \"column\": \"TenureGroup\",\n
\"properties\": {\n  \"dtype\": \"category\",\n
\"num_unique_values\": 7,\n  \"samples\": [\n  \"0-
12\",\n  \"13-24\",\n  \"61-72\"\n  ],\n
\"semantic_type\": \"\",\n  \"description\": \"\"\n  }\
n  },\n  {\n  \"column\": \"No\",\n  \"properties\": {\n
\"dtype\": \"number\",\n  \"std\": 246,\n  \"min\": 356,\n
\"max\": 1070,\n  \"num_unique_values\": 7,\n
\"samples\": [\n  1070,\n  738,\n  1028\n
],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n
}\n  },\n  {\n  \"column\": \"Yes\",\n  \"properties\": {\
n  \"dtype\": \"number\",\n  \"std\": 335,\n
\"min\": 6,\n  \"max\": 999,\n  \"num_unique_values\": 7,\
n  \"samples\": [\n  999,\n  309,\n  93\
n  ],\n  \"semantic_type\": \"\",\n
\"description\": \"\"\n  }\n  },\n  {\n  \"column\":
\"Total\",\n  \"properties\": {\n  \"dtype\": \"number\",\n
\"std\": 528,\n  \"min\": 362,\n  \"max\": 2069,\n
\"num_unique_values\": 7,\n  \"samples\": [\n  2069,\n
1047,\n  1121\n  ],\n  \"semantic_type\": \"\",\n
\"description\": \"\"\n  }\n  },\n  {\n  \"column\":
\"ChurnRate\",\n  \"properties\": {\n  \"dtype\":
\"number\",\n  \"std\": 15.224086311360763,\n  \"min\":
1.6574585635359116,\n  \"max\": 48.28419526341228,\n
\"num_unique_values\": 7,\n  \"samples\": [\n
48.28419526341228,\n  29.512893982808023,\n
```

8.296164139161464\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   }\n]\
n}","type":"dataframe","variable_name":"tenure_cohort_churn"}

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12, 7))
sns.barplot(x=tenure_cohort_churn.index, y='ChurnRate',
data=tenure_cohort_churn, palette='rocket',
hue=tenure_cohort_churn.index, legend=False)
plt.title('Churn Rate by Tenure-Based Cohort')
plt.xlabel('Tenure Group (months)')
plt.ylabel('Churn Rate (%)')
plt.ylim(0, 100) # Ensure y-axis goes from 0 to 100
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```



Customer Segmentation by Engagement Score

To create customer segments, we will use the `engagement_score` calculated previously. A simple yet effective method is to divide customers into groups (e.g., Low, Medium, High) based on the quantiles of this score.

```
# Define the number of segments and their labels
num_segments = 3
```

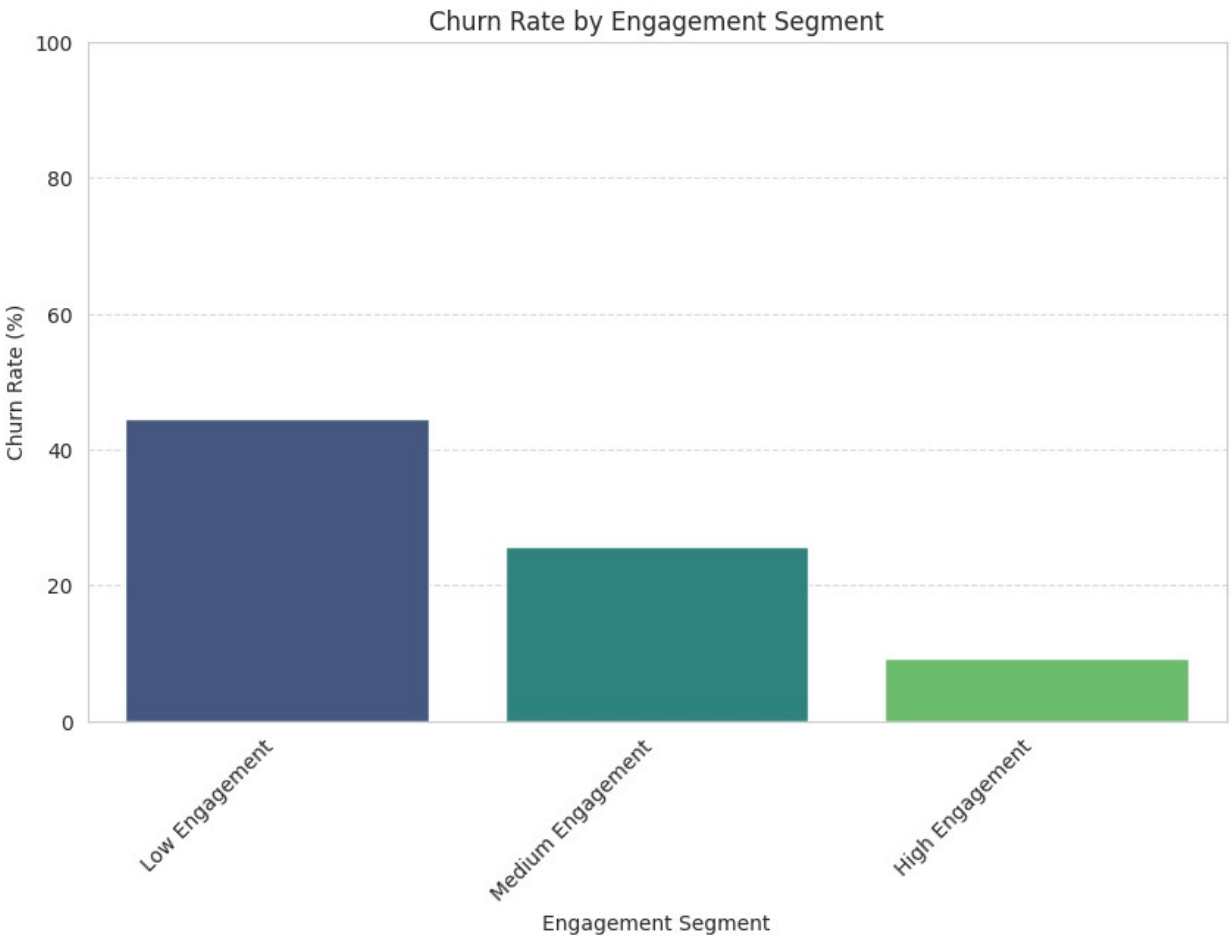
```
segment_labels = ['Low Engagement', 'Medium Engagement', 'High
Engagement']

# Create engagement segments using pd.qcut (quantile-based)
df['EngagementSegment'] = pd.qcut(df['engagement_score'],
q=num_segments, labels=segment_labels, precision=2)

print("Distribution of customers across Engagement Segments:")
display(df['EngagementSegment'].value_counts().sort_index())

# Calculate churn rate for each engagement segment
engagement_churn = df.groupby(['EngagementSegment', 'Churn'],
observed=False).size().unstack(fill_value=0)
engagement_churn['Total'] = engagement_churn['Yes'] +
engagement_churn['No']
engagement_churn['ChurnRate'] = (engagement_churn['Yes'] /
engagement_churn['Total']) * 100

print("\nChurn Rate by Engagement Segment:")
display(engagement_churn)
```

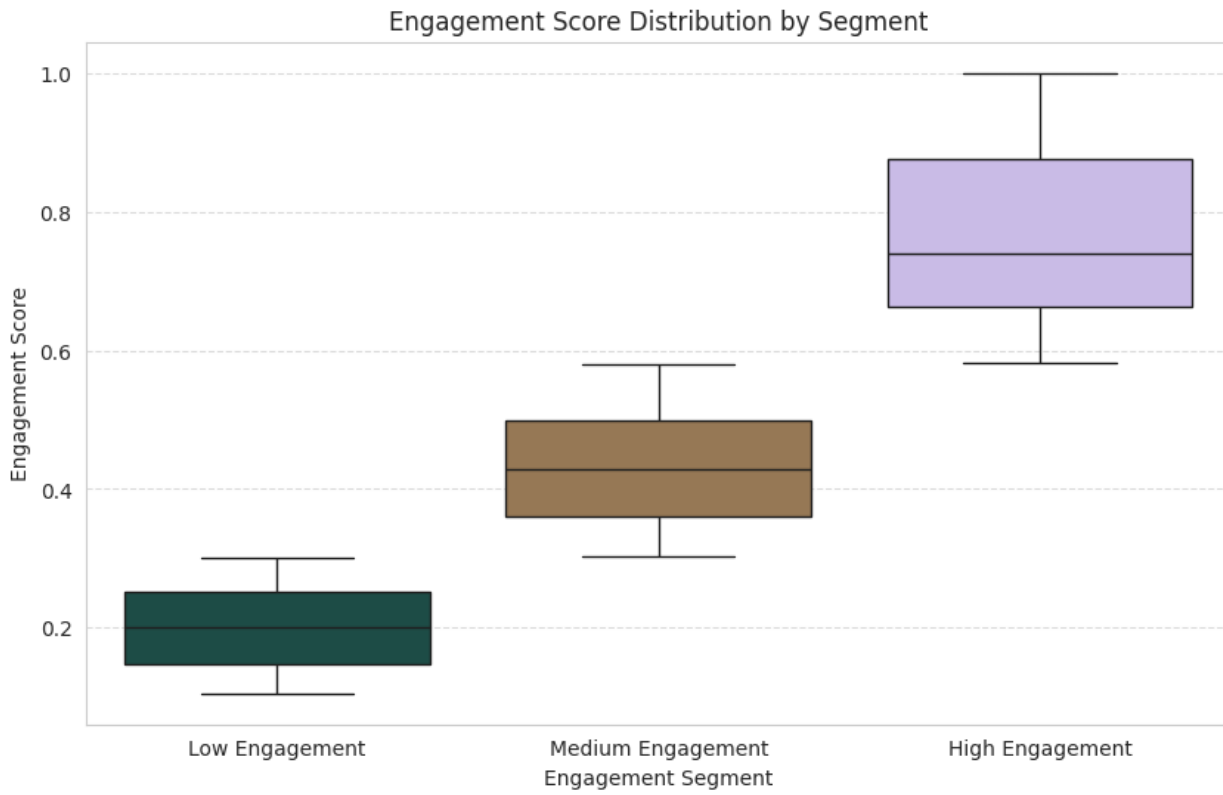
```
Distribution of customers across Engagement Segments:

EngagementSegment
Low Engagement       2349
Medium Engagement    2346
High Engagement      2348
Name: count, dtype: int64


Churn Rate by Engagement Segment:
```


```
{"summary":"{\n  \"name\": \"engagement_churn\",\n  \"rows\": 3,\n
\"fields\": [\n    {\n      \"column\": \"EngagementSegment\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 3,\n        \"samples\": [\n          \"Low
Engagement\",\n          \"Medium Engagement\",\n          \"High
Engagement\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"No\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 412,\n        \"min\": 1304,\n        \"max\": 2129,\n
\"num_unique_values\": 3,\n        \"samples\": [\n          1304,\n
1741,\n          2129\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Yes\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 413,\n        \"min\": 219,\n        \"max\": 1045,\n
\"num_unique_values\": 3,\n        \"samples\": [\n          1045,\n
605,\n          219\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Total\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 1,\n        \"min\": 2346,\n        \"max\": 2349,\n
```

\"num_unique_values\": 3,\n   \"samples\": [\n   2349,\n
2346,\n   2348\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"ChurnRate\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 17.59182039959163,\n   \"min\":
9.327086882453152,\n   \"max\": 44.48701575138357,\n
\"num_unique_values\": 3,\n   \"samples\": [\n
44.48701575138357,\n   25.788576300085253,\n
9.327086882453152\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   }\n]\
n}","type":"dataframe","variable_name":"engagement_churn"}

```
import matplotlib.pyplot as plt
import seaborn as sns

# Visualize churn rate by engagement segment
plt.figure(figsize=(10, 6))
sns.barplot(x=engagement_churn.index, y='ChurnRate',
data=engagement_churn, palette='viridis', hue=engagement_churn.index,
legend=False)
plt.title('Churn Rate by Engagement Segment')
plt.xlabel('Engagement Segment')
plt.ylabel('Churn Rate (%)')
plt.ylim(0, 100)
plt.xticks(rotation=45, ha='right')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()

# Visualize the distribution of engagement scores within each segment
plt.figure(figsize=(10, 6))
sns.boxplot(x='EngagementSegment', y='engagement_score', data=df,
palette='cubehelix', hue='EngagementSegment', legend=False)
plt.title('Engagement Score Distribution by Segment')
plt.xlabel('Engagement Segment')
plt.ylabel('Engagement Score')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```


Churn Rate by Engagement Segment
100
80
60
40
20
0
Churn Rate (%)
Low Engagement
Medium Engagement
High Engagement
Engagement Segment



Average Monthly Charges by Engagement Segment

Let's analyze the average monthly charges for each of the engagement segments we've created. This will help us understand the spending patterns of customers at different engagement levels.

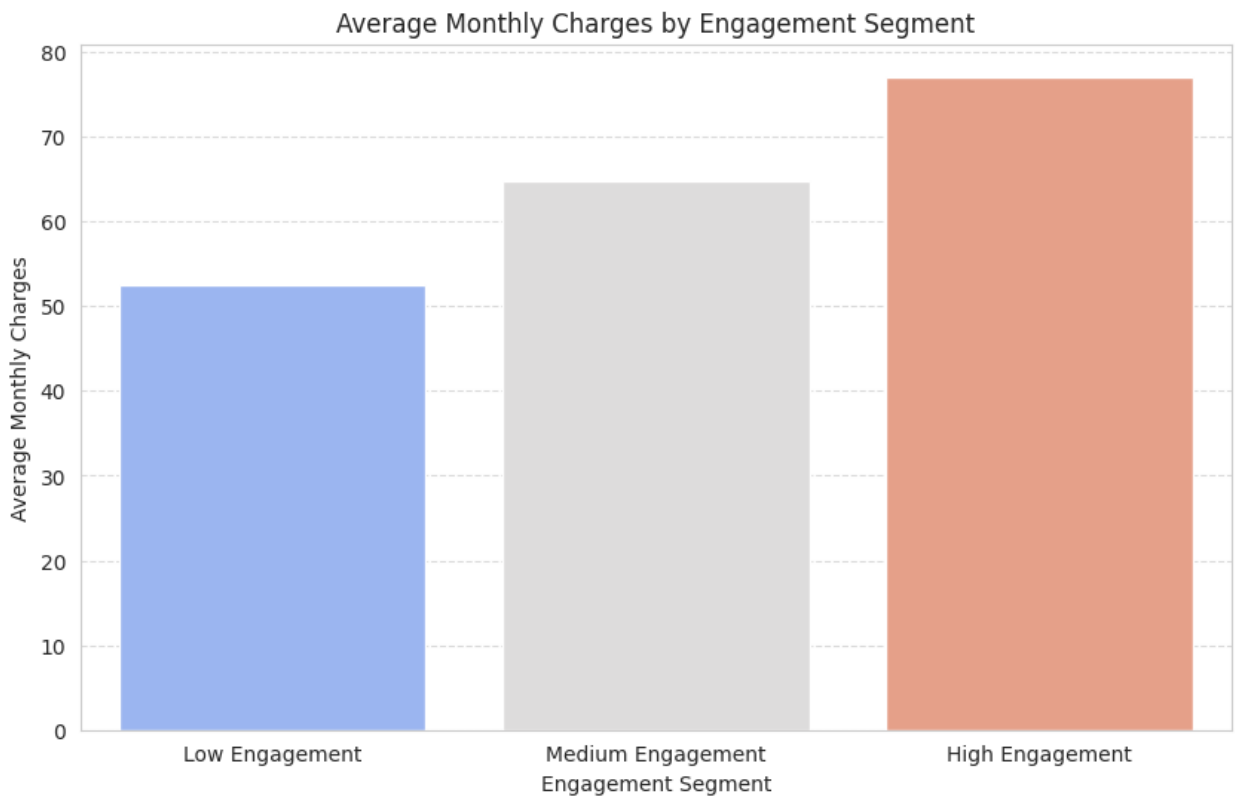
```
# Calculate average monthly charges by engagement segment
average_monthly_charges_by_segment = df.groupby('EngagementSegment',
observed=False)['MonthlyCharges'].mean().reset_index()

print("Average Monthly Charges by Engagement Segment:")
display(average_monthly_charges_by_segment)
```

```
Average Monthly Charges by Engagement Segment:
```


```
{"summary":"{\n  \"name\": \"average_monthly_charges_by_segment\",\n
\"rows\": 3,\n  \"fields\": [\n    {\n      \"column\":
\"EngagementSegment\",\n      \"properties\": {\n        \"dtype\":
\"category\",\n        \"num_unique_values\": 3,\n        \"samples\":
[\n          \"Low Engagement\",\n          \"Medium Engagement\",\n
\"High Engagement\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"MonthlyCharges\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 12.310304973359397,\n        \"min\":
52.44397616006811,\n        \"max\": 77.06456558773425,\n
\"num_unique_values\": 3,\n        \"samples\": [\n
52.44397616006811,\n          64.78179880647912,\n
```

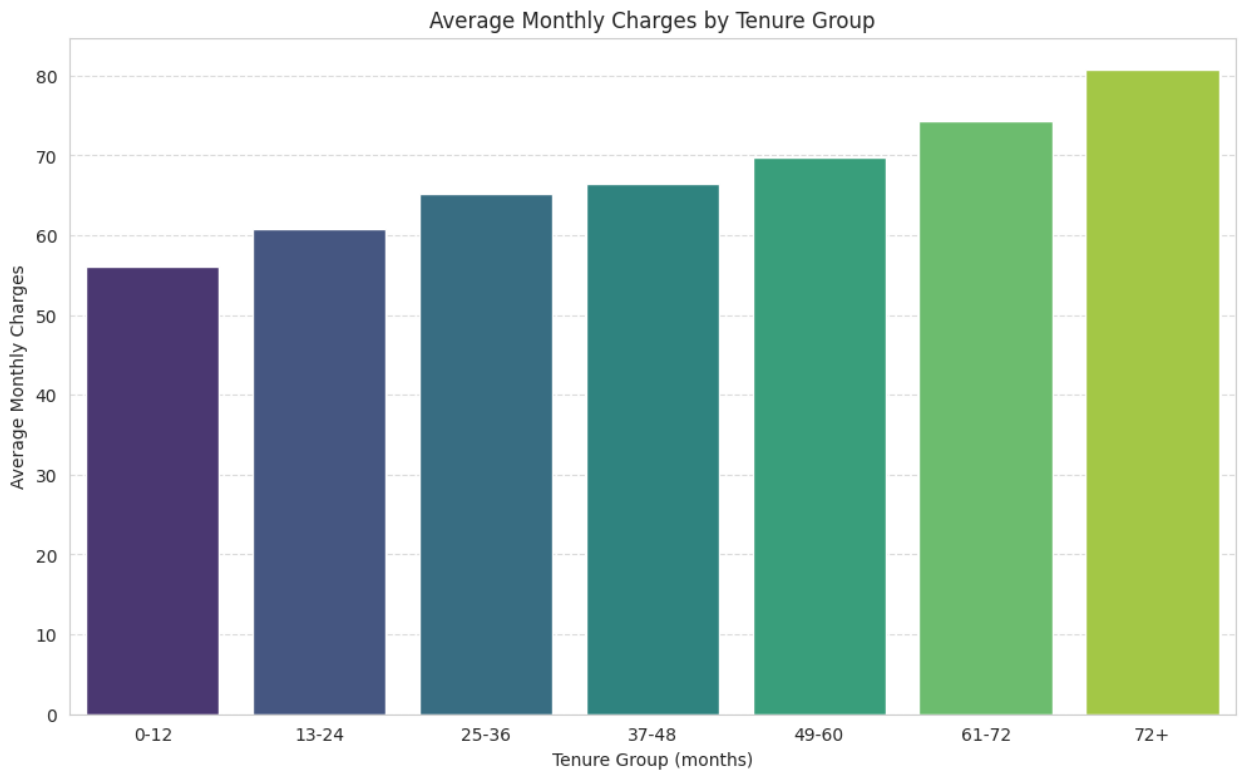
77.06456558773425\n   ],\n   \"semantic_type\": \"\",\n \"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe","variable_name":"average_monthly_charges_by_segment"}

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 6))
sns.barplot(x='EngagementSegment', y='MonthlyCharges',
data=average_monthly_charges_by_segment, palette='coolwarm',
hue='EngagementSegment', legend=False)
plt.title('Average Monthly Charges by Engagement Segment')
plt.xlabel('Engagement Segment')
plt.ylabel('Average Monthly Charges')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```




```
plt.figure(figsize=(12, 7))
sns.barplot(x='TenureGroup', y='MonthlyCharges',
data=average_monthly_charges_by_tenure, palette='viridis',
hue='TenureGroup', legend=False)
plt.title('Average Monthly Charges by Tenure Group')
plt.xlabel('Tenure Group (months)')
plt.ylabel('Average Monthly Charges')
```

```
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```



Machine Learning Model Development

To build a predictive model for customer churn, we need to preprocess the data. This involves several steps:

1. **Drop Irrelevant Features**: Remove `customerID` as it's just an identifier.
2. **Handle Missing Values and Convert Data Types**: The `TotalCharges` column is currently an object type and contains some missing values (represented as empty strings), which need to be converted to a numeric type after handling these.
3. **Encode Target Variable**: Convert the `Churn` column ('Yes'/'No') into numerical format (1/0).
4. **One-Hot Encode Categorical Features**: Transform categorical features into a numerical format that machine learning models can understand.
5. **Scale Numerical Features**: Standardize numerical features to ensure that no single feature dominates the model due to its scale.
6. **Split Data**: Divide the dataset into training and testing sets for model development and evaluation.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
```

```python
import numpy as np

# Drop customerID column as it's not useful for modeling
df_model = df.drop('customerID', axis=1).copy()

# Handle 'TotalCharges' column
# Replace empty strings with NaN and convert to numeric
df_model['TotalCharges'] = df_model['TotalCharges'].replace(' ',
np.nan).astype(float)
# Fill missing 'TotalCharges' with the median (or mean, depending on
distribution)
df_model['TotalCharges'] =
df_model['TotalCharges'].fillna(df_model['TotalCharges'].median())

# Convert 'Churn' column to numerical (0 for No, 1 for Yes)
df_model['Churn'] = df_model['Churn'].map({'No': 0, 'Yes': 1})

# Separate target variable 'y' and features 'X'
X = df_model.drop('Churn', axis=1)
y = df_model['Churn']

# Identify categorical and numerical columns
categorical_features = X.select_dtypes(include=['object']).columns
numerical_features = X.select_dtypes(include=['number']).columns

print(f"Categorical features: {list(categorical_features)}")
print(f"Numerical features: {list(numerical_features)}")
```

```
Categorical features: ['gender', 'Partner', 'Dependents',
'PhoneService', 'MultipleLines', 'InternetService', 'OnlineSecurity',
'OnlineBackup', 'DeviceProtection', 'TechSupport', 'StreamingTV',
'StreamingMovies', 'Contract', 'PaperlessBilling', 'PaymentMethod']
Numerical features: ['SeniorCitizen', 'tenure', 'MonthlyCharges',
'TotalCharges']
```

```python
# Create preprocessing pipelines for numerical and categorical
features
numerical_transformer = StandardScaler()
categorical_transformer = OneHotEncoder(handle_unknown='ignore')

# Create a column transformer to apply different transformations to
different columns
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numerical_transformer, numerical_features),
        ('cat', categorical_transformer, categorical_features)
    ])

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
```

```
test_size=0.2, random_state=42, stratify=y)

print(f"Shape of X_train: {X_train.shape}")
print(f"Shape of X_test: {X_test.shape}")
print(f"Shape of y_train: {y_train.shape}")
print(f"Shape of y_test: {y_test.shape}")
```

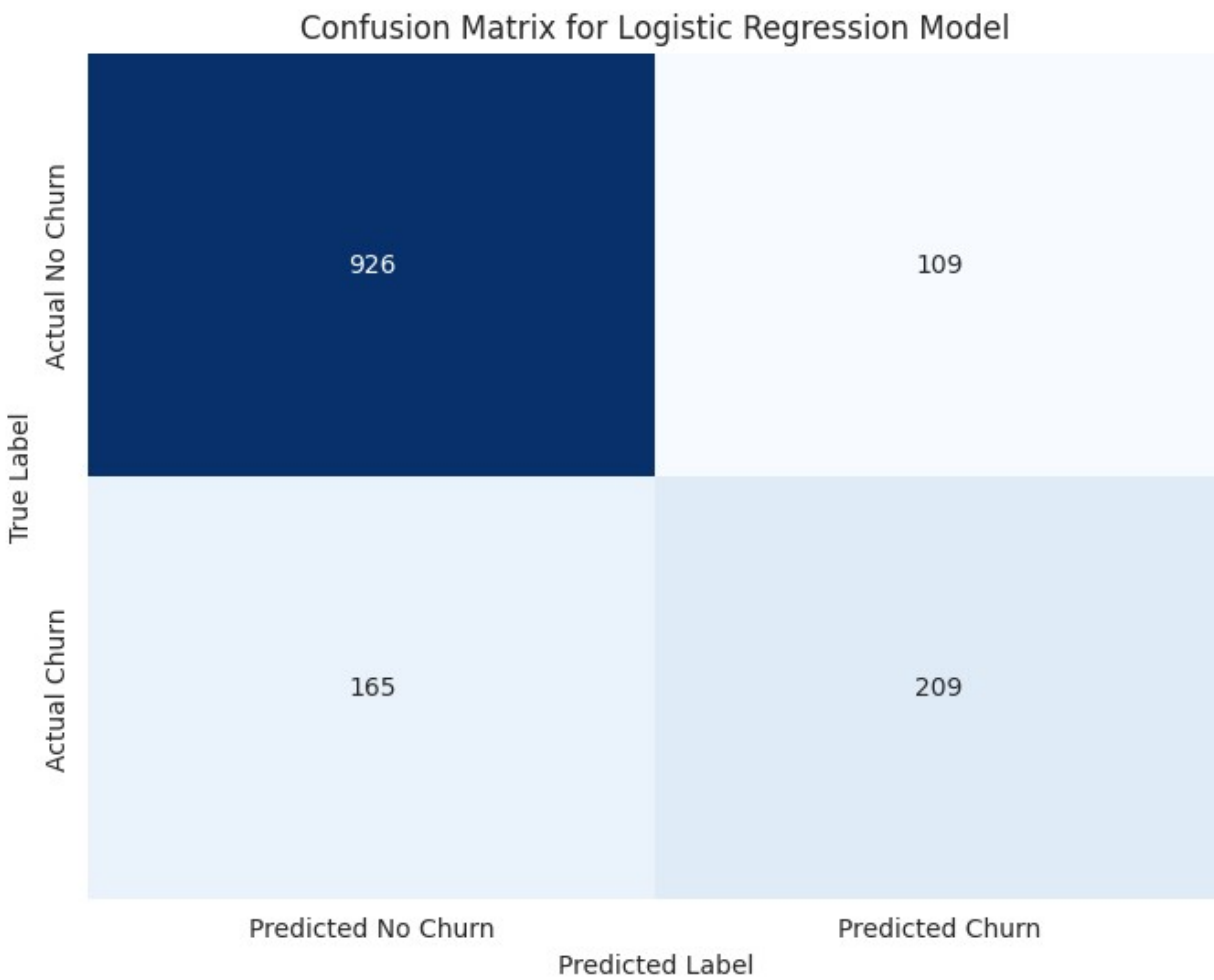
```
Shape of X_train: (5634, 19)
Shape of X_test: (1409, 19)
Shape of y_train: (5634,)
Shape of y_test: (1409,)
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score, roc_auc_score

# Create the full pipeline with preprocessing and a Logistic
Regression model
model = Pipeline(steps=[('preprocessor', preprocessor),
                        ('classifier',
LogisticRegression(solver='liblinear', random_state=42))])

# Train the model
model.fit(X_train, y_train)

# Make predictions on the test set
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
roc_auc = roc_auc_score(y_test, y_prob)

print(f"Model Accuracy: {accuracy:.4f}")
print(f"Model Precision: {precision:.4f}")
print(f"Model Recall: {recall:.4f}")
print(f"Model F1-Score: {f1:.4f}")
print(f"Model ROC AUC: {roc_auc:.4f}")
```

```
Model Accuracy: 0.8055
Model Precision: 0.6572
Model Recall: 0.5588
Model F1-Score: 0.6040
Model ROC AUC: 0.8419
```

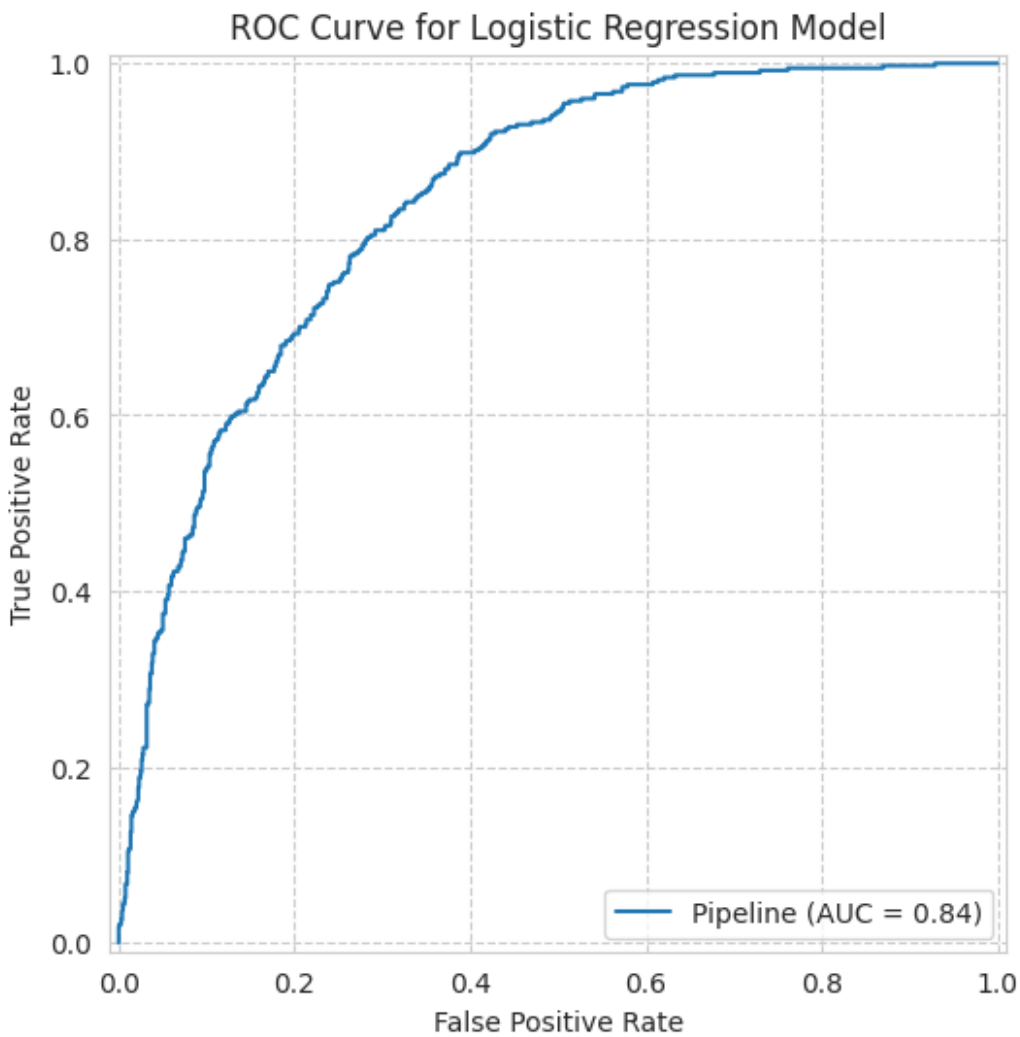
```
from sklearn.metrics import confusion_matrix

# Calculate the confusion matrix
```

```
cm = confusion_matrix(y_test, y_pred)

# Display the confusion matrix using a heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False,
            xticklabels=['Predicted No Churn', 'Predicted Churn'],
            yticklabels=['Actual No Churn', 'Actual Churn'])
plt.title('Confusion Matrix for Logistic Regression Model')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```




```
from sklearn.metrics import RocCurveDisplay

plt.figure(figsize=(8, 6))
RocCurveDisplay.from_estimator(model, X_test, y_test, ax=plt.gca())
plt.title('ROC Curve for Logistic Regression Model')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
```

```
plt.grid(linestyle='--')
plt.show()
```




```
import pandas as pd

# Get feature names from OneHotEncoder for categorical features
cat_feature_names =
model.named_steps['preprocessor'].named_transformers_['cat'].get_featu
re_names_out(categorical_features)

# Combine numerical and categorical feature names
all_feature_names = list(numerical_features) + list(cat_feature_names)

# Get coefficients from the Logistic Regression model
coefficients = model.named_steps['classifier'].coef_[0]

# Create a DataFrame for feature importance
```

```
feature_importance = pd.DataFrame({'Feature': all_feature_names,
'Coefficient': coefficients})

# Sort by the absolute value of coefficients to see most impactful
features
feature_importance['Absolute_Coefficient'] =
abs(feature_importance['Coefficient'])
feature_importance =
feature_importance.sort_values(by='Absolute_Coefficient',
ascending=False)

print("Feature Importance (Logistic Regression Coefficients):")
display(feature_importance.head(15)) # Display top 15 features
```

```
Feature Importance (Logistic Regression Coefficients):
```


```
{"summary":"{\n  \"name\": \"display(feature_importance\",\n
\"rows\": 15,\n  \"fields\": [\n  {\n  \"column\": \"Feature\",\
n  \"properties\": {\n  \"dtype\": \"string\",\n
\"num_unique_values\": 15,\n  \"samples\": [\n
\"InternetService_No\",\n  \"TechSupport_No internet
service\",\n  \"tenure\"\n  ],\n
\"semantic_type\": \"\",\n  \"description\": \"\"\n  }\
n  },\n  {\n  \"column\": \"Coefficient\",\n
\"properties\": {\n  \"dtype\": \"number\",\n  \"std\":
0.5020985410200768,\n  \"min\": -1.2360739430751668,\n
\"max\": 0.6083785820561477,\n  \"num_unique_values\": 10,\n
\"samples\": [\n  -0.3059445521550298,\n  -
0.7860378784220717,\n  -0.5560171006008583\n  ],\n
\"semantic_type\": \"\",\n  \"description\": \"\"\n  }\
n  },\n  {\n  \"column\": \"Absolute_Coefficient\",\n
\"properties\": {\n  \"dtype\": \"number\",\n  \"std\":
0.26035139473753993,\n  \"min\": 0.3059445521550298,\n
\"max\": 1.2360739430751668,\n  \"num_unique_values\": 10,\n
\"samples\": [\n  0.3059445521550298,\n
0.7860378784220717,\n  0.5560171006008583\n  ],\n
\"semantic_type\": \"\",\n  \"description\": \"\"\n  }\
n  }\n  ]\n}","type":"dataframe"}
```


Hyperparameter Tuning with GridSearchCV

To further optimize the Logistic Regression model, we'll perform hyperparameter tuning using `GridSearchCV`. This process systematically works through multiple combinations of parameter values, cross-validating each combination to determine which yields the best performance.

We will tune the following parameters for the `LogisticRegression` classifier:

- `penalty`: The type of regularization (`l1`, `l2`, `elasticnet`, `none`). `l1` and `l2` are common choices.

- `C`: Inverse of regularization strength. Smaller values specify stronger regularization.
- `solver`: Algorithm to use in the optimization problem. Different solvers work best with different penalties.

```
from sklearn.model_selection import GridSearchCV

# Define the parameter grid for Logistic Regression
param_grid = {
    'classifier__penalty': ['l1', 'l2'],
    'classifier__C': [0.001, 0.01, 0.1, 1, 10, 100],
    'classifier__solver': ['liblinear', 'saga']  # 'liblinear' and
'saga' support 'l1' penalty
}

# Create a GridSearchCV object
grid_search = GridSearchCV(model, param_grid, cv=5,
scoring='accuracy', n_jobs=-1, verbose=1)

# Fit GridSearchCV to the training data
grid_search.fit(X_train, y_train)

# Print the best parameters and best score
print(f"Best parameters found: {grid_search.best_params_}")
print(f"Best cross-validation accuracy:
{grid_search.best_score_:.4f}")

# Get the best model from grid search
best_model = grid_search.best_estimator_

# Make predictions with the best model on the test set
y_pred_tuned = best_model.predict(X_test)
y_prob_tuned = best_model.predict_proba(X_test)[:, 1]

# Evaluate the tuned model
accuracy_tuned = accuracy_score(y_test, y_pred_tuned)
precision_tuned = precision_score(y_test, y_pred_tuned)
recall_tuned = recall_score(y_test, y_pred_tuned)
f1_tuned = f1_score(y_test, y_pred_tuned)
roc_auc_tuned = roc_auc_score(y_test, y_prob_tuned)

print("\nTuned Model Performance:")
print(f"Model Accuracy (Tuned): {accuracy_tuned:.4f}")
print(f"Model Precision (Tuned): {precision_tuned:.4f}")
print(f"Model Recall (Tuned): {recall_tuned:.4f}")
print(f"Model F1-Score (Tuned): {f1_tuned:.4f}")
print(f"Model ROC AUC (Tuned): {roc_auc_tuned:.4f}")
```

```
Fitting 5 folds for each of 24 candidates, totalling 120 fits
Best parameters found: {'classifier__C': 0.01, 'classifier__penalty':
'l2', 'classifier__solver': 'liblinear'}
Best cross-validation accuracy: 0.8048
```

```
Tuned Model Performance:
Model Accuracy (Tuned): 0.7984
Model Precision (Tuned): 0.6552
Model Recall (Tuned): 0.5080
Model F1-Score (Tuned): 0.5723
Model ROC AUC (Tuned): 0.8388
```

Random Forest Classifier for Churn Prediction

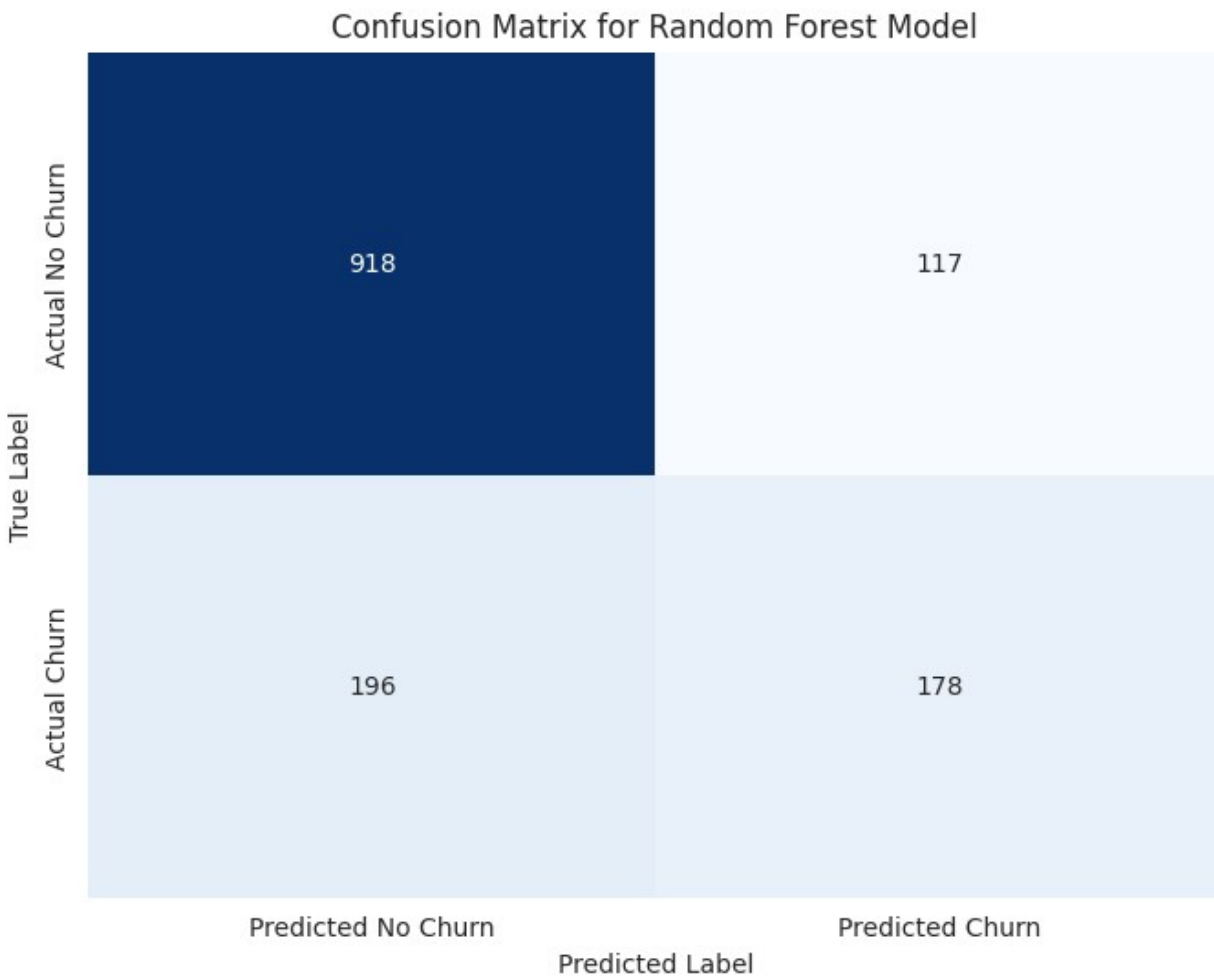
To compare with the Logistic Regression model, we will now train a Random Forest Classifier. Random Forests are ensemble learning methods that can often provide better accuracy and handle non-linear relationships in data. We will use a similar pipeline for preprocessing and then evaluate its performance.

```
from sklearn.ensemble import RandomForestClassifier

# Create the full pipeline with preprocessing and a Random Forest
Classifier model
rf_model = Pipeline(steps=[('preprocessor', preprocessor),
                           ('classifier',
RandomForestClassifier(random_state=42))])

# Train the Random Forest model
rf_model.fit(X_train, y_train)

# Make predictions on the test set
y_pred_rf = rf_model.predict(X_test)
y_prob_rf = rf_model.predict_proba(X_test)[:, 1]

# Evaluate the Random Forest model
accuracy_rf = accuracy_score(y_test, y_pred_rf)
precision_rf = precision_score(y_test, y_pred_rf)
recall_rf = recall_score(y_test, y_pred_rf)
f1_rf = f1_score(y_test, y_pred_rf)
roc_auc_rf = roc_auc_score(y_test, y_prob_rf)

print("\nRandom Forest Model Performance:")
print(f"Model Accuracy (Random Forest): {accuracy_rf:.4f}")
print(f"Model Precision (Random Forest): {precision_rf:.4f}")
print(f"Model Recall (Random Forest): {recall_rf:.4f}")
print(f"Model F1-Score (Random Forest): {f1_rf:.4f}")
print(f"Model ROC AUC (Random Forest): {roc_auc_rf:.4f}")
```

```
Random Forest Model Performance:
Model Accuracy (Random Forest): 0.7779
Model Precision (Random Forest): 0.6034
Model Recall (Random Forest): 0.4759
```

```
Model F1-Score (Random Forest): 0.5321
Model ROC AUC (Random Forest): 0.8164
```

```python
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# Calculate the confusion matrix for the Random Forest model
cm_rf = confusion_matrix(y_test, y_pred_rf)

# Display the confusion matrix using a heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(cm_rf, annot=True, fmt='d', cmap='Blues', cbar=False,
            xticklabels=['Predicted No Churn', 'Predicted Churn'],
            yticklabels=['Actual No Churn', 'Actual Churn'])
plt.title('Confusion Matrix for Random Forest Model')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```



Hyperparameter Tuning for Random Forest Classifier

To further optimize the Random Forest Classifier, we will perform hyperparameter tuning using `GridSearchCV`. This process will explore different combinations of key hyperparameters to find the configuration that yields the best performance.

```
from sklearn.model_selection import GridSearchCV

# Define the parameter grid for Random Forest
# Limiting the number of parameters to keep computation time
reasonable for demonstration
param_grid_rf = {
    'classifier__n_estimators': [100, 200],
    'classifier__max_depth': [10, 20, None],
    'classifier__min_samples_leaf': [1, 2],
    'classifier__min_samples_split': [2, 5]
}

# Create a GridSearchCV object for Random Forest
grid_search_rf = GridSearchCV(rf_model, param_grid_rf, cv=3,
scoring='accuracy', n_jobs=-1, verbose=1)

# Fit GridSearchCV to the training data
grid_search_rf.fit(X_train, y_train)

# Print the best parameters and best score
print(f"Best parameters for Random Forest found:
{grid_search_rf.best_params_}")
print(f"Best cross-validation accuracy for Random Forest:
{grid_search_rf.best_score_:.4f}")

# Get the best Random Forest model from grid search
best_rf_model = grid_search_rf.best_estimator_

# Make predictions with the best Random Forest model on the test set
y_pred_rf_tuned = best_rf_model.predict(X_test)
y_prob_rf_tuned = best_rf_model.predict_proba(X_test)[:, 1]

# Evaluate the tuned Random Forest model
accuracy_rf_tuned = accuracy_score(y_test, y_pred_rf_tuned)
precision_rf_tuned = precision_score(y_test, y_pred_rf_tuned)
recall_rf_tuned = recall_score(y_test, y_pred_rf_tuned)
f1_rf_tuned = f1_score(y_test, y_pred_rf_tuned)
roc_auc_rf_tuned = roc_auc_score(y_test, y_prob_rf_tuned)

print("\nTuned Random Forest Model Performance:")
print(f"Model Accuracy (Tuned RF): {accuracy_rf_tuned:.4f}")
print(f"Model Precision (Tuned RF): {precision_rf_tuned:.4f}")
print(f"Model Recall (Tuned RF): {recall_rf_tuned:.4f}")
```

```
print(f"Model F1-Score (Tuned RF): {f1_rf_tuned:.4f}")
print(f"Model ROC AUC (Tuned RF): {roc_auc_rf_tuned:.4f}")
```

```
Fitting 3 folds for each of 24 candidates, totalling 72 fits
Best parameters for Random Forest found: {'classifier__max_depth': 10,
'classifier__min_samples_leaf': 2, 'classifier__min_samples_split': 2,
'classifier__n_estimators': 100}
Best cross-validation accuracy for Random Forest: 0.8042

Tuned Random Forest Model Performance:
Model Accuracy (Tuned RF): 0.8084
Model Precision (Tuned RF): 0.6844
Model Recall (Tuned RF): 0.5160
Model F1-Score (Tuned RF): 0.5884
Model ROC AUC (Tuned RF): 0.8393
```

Updated Comparison of Model Performance

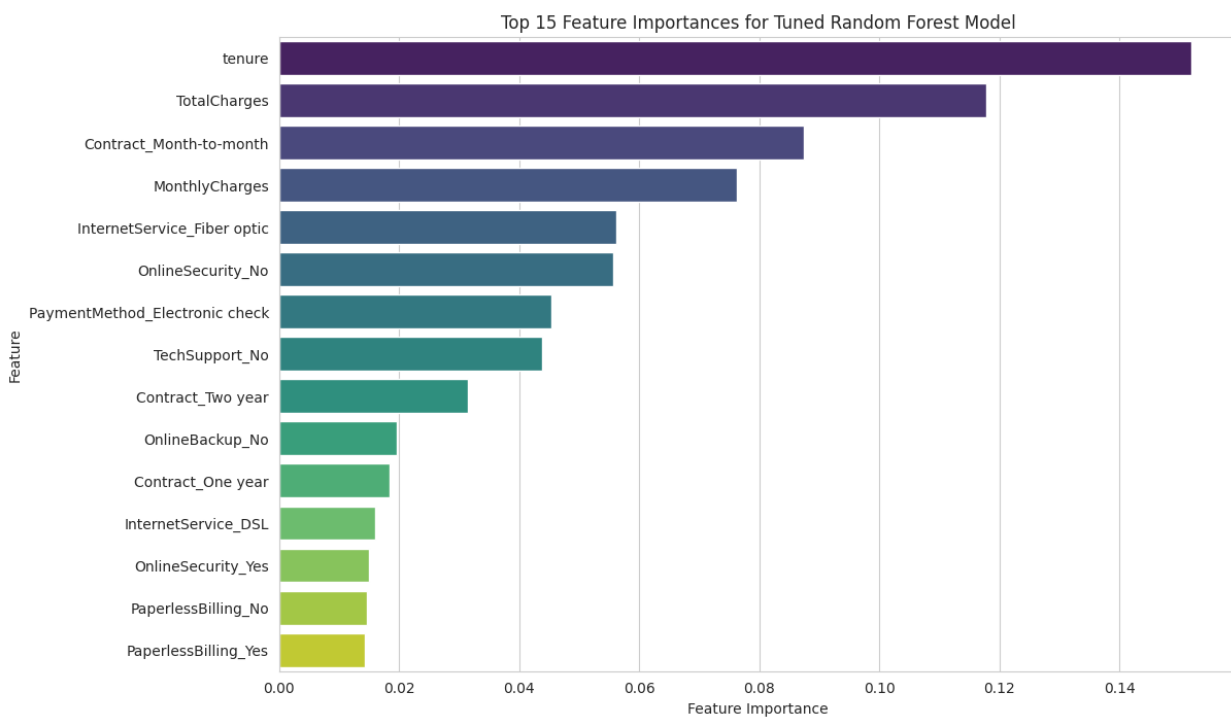
Now, let's compare the performance metrics of the Logistic Regression model (tuned) and the newly tuned Random Forest Classifier.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Get feature names from OneHotEncoder for categorical features
cat_feature_names_rf =
best_rf_model.named_steps['preprocessor'].named_transformers_['cat'].g
et_feature_names_out(categorical_features)

# Combine numerical and categorical feature names
all_feature_names_rf = list(numerical_features) +
list(cat_feature_names_rf)

# Get feature importances from the Random Forest model
# The classifier is the second step in the pipeline
feature_importances_rf =
best_rf_model.named_steps['classifier'].feature_importances_

# Create a DataFrame for feature importance
feature_importance_df_rf = pd.DataFrame({'Feature':
all_feature_names_rf, 'Importance': feature_importances_rf})

# Sort by importance to see most impactful features
feature_importance_df_rf =
feature_importance_df_rf.sort_values(by='Importance', ascending=False)

print("Feature Importance (Random Forest):")
display(feature_importance_df_rf.head(15)) # Display top 15 features
```

```
Feature Importance (Random Forest):
```

{"summary":"{\n \"name\": \"display(feature_importance_df_rf\",\n \"rows\": 15,\n \"fields\": [\n {\n \"column\": \"Feature\",\n \"properties\": {\n \"dtype\": \"string\",\n \"num_unique_values\": 15,\n \"samples\": [\n \"OnlineBackup_No\",\n \"InternetService_DSL\",\n \"tenure\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Importance\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0.041752051126435354,\n \"min\": 0.014341840033168914,\n \"max\": 0.15208172222311517,\n \"num_unique_values\": 15,\n \"samples\": [\n 0.019592484460864174,\n 0.016026852581109557,\n 0.15208172222311517\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n }\n]\n}","type":"dataframe"}

```
plt.figure(figsize=(12, 8))
sns.barplot(x='Importance', y='Feature',
data=feature_importance_df_rf.head(15), palette='viridis',
hue='Feature', legend=False)
plt.title('Top 15 Feature Importances for Tuned Random Forest Model')
plt.xlabel('Feature Importance')
plt.ylabel('Feature')
plt.show()
```

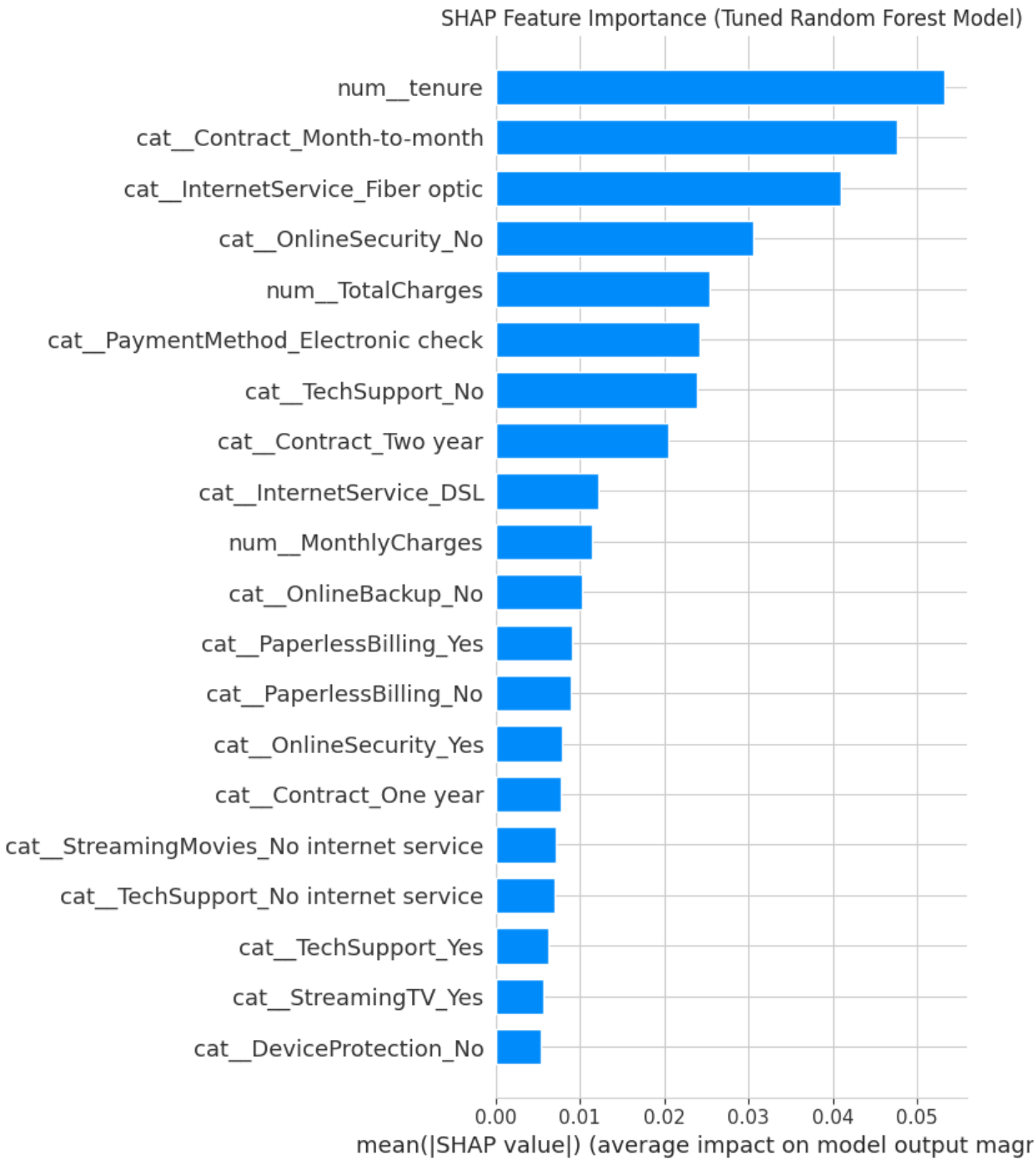


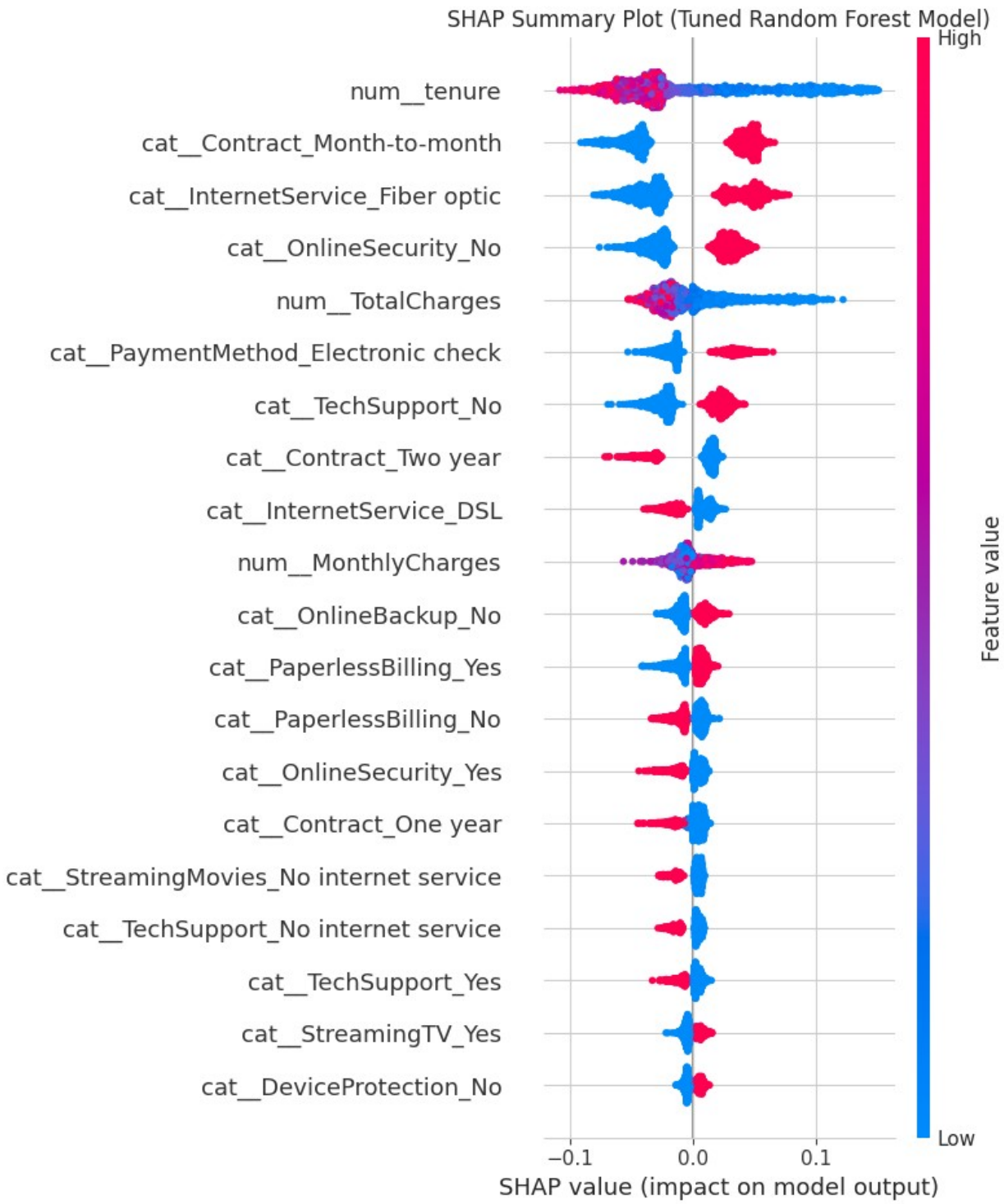
SHAP Summary Plot for Model Explainability

SHAP (SHapley Additive exPlanations) is a game theory approach to explain the output of any machine learning model. It connects optimal credit allocation with local explanations using the

classic Shapley values from game theory and their related extensions. This summary plot helps to understand the global impact of features on the model's predictions.

```
import shap

# Create a SHAP explainer object for the best Random Forest model
# We use the preprocessed X_train data to train the explainer
explainer =
shap.TreeExplainer(best_rf_model.named_steps['classifier'])

# Get the preprocessed X_test data
X_test_processed =
best_rf_model.named_steps['preprocessor'].transform(X_test)

# SHAP values for the test set
shap_values = explainer.shap_values(X_test_processed)

# Get the feature names directly from the fitted preprocessor
# This ensures the number and order of names match the preprocessed
data
feature_names_for_shap =
best_rf_model.named_steps['preprocessor'].get_feature_names_out()

# Create a DataFrame for the preprocessed test data with correct
column names
X_test_processed_df = pd.DataFrame(X_test_processed,
columns=feature_names_for_shap)

# Plot the SHAP summary plot (using shap_values for class 1)
plt.figure(figsize=(12, 8))
shap.summary_plot(shap_values[:, :, 1], X_test_processed_df,
plot_type="bar", show=False)
plt.title('SHAP Feature Importance (Tuned Random Forest Model)')
plt.tight_layout()
plt.show()

plt.figure(figsize=(12, 8))
shap.summary_plot(shap_values[:, :, 1], X_test_processed_df,
show=False)
plt.title('SHAP Summary Plot (Tuned Random Forest Model)')
plt.tight_layout()
plt.show()
```


SHAP Feature Importance (Tuned Random Forest Model)
num__tenure
cat__Contract_Month-to-month
cat__InternetService_Fiber optic
cat__OnlineSecurity_No
num__TotalCharges
cat__PaymentMethod_Electronic check
cat__TechSupport_No
cat__Contract_Two year
cat__InternetService_DSL
num__MonthlyCharges
cat__OnlineBackup_No
cat__PaperlessBilling_Yes
cat__PaperlessBilling_No
cat__OnlineSecurity_Yes
cat__Contract_One year
cat__StreamingMovies_No internet service
cat__TechSupport_No internet service
cat__TechSupport_Yes
cat__StreamingTV_Yes
cat__DeviceProtection_No
0.00
0.01
0.02
0.03
0.04
0.05
mean(|SHAP value|) (average impact on model output magr

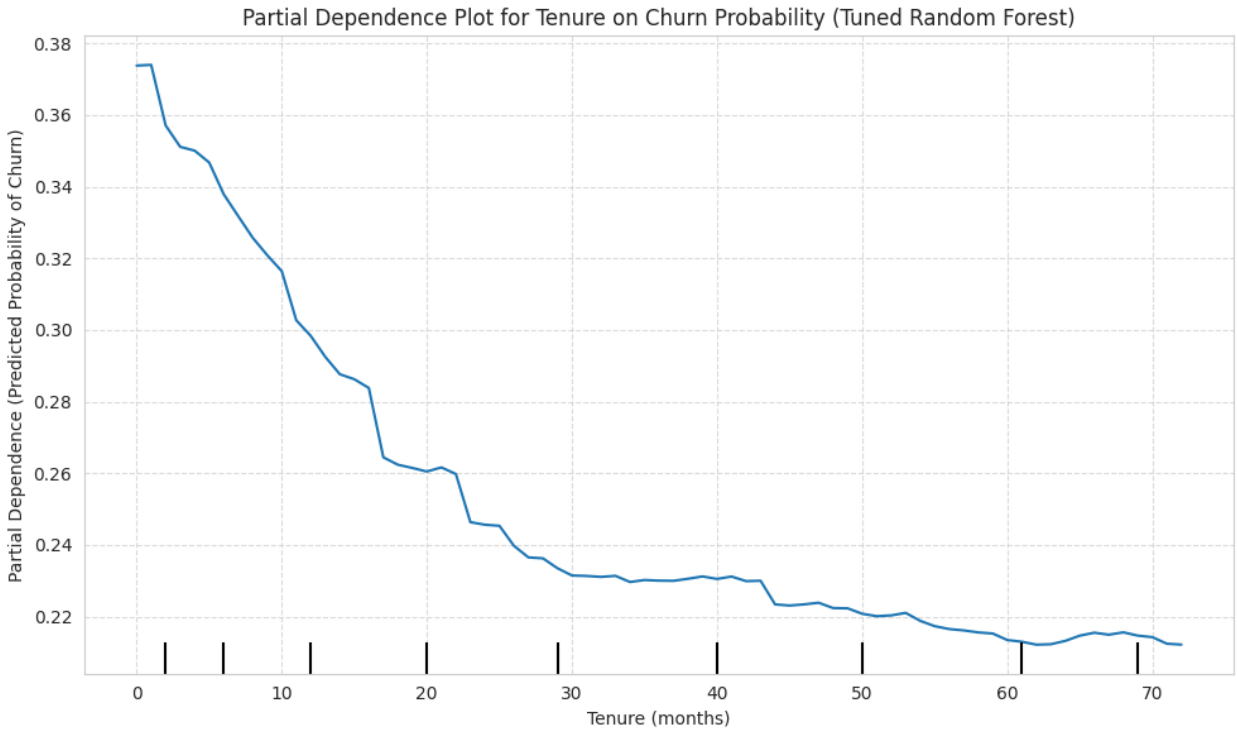


Partial Dependence Plot for 'tenure'

A Partial Dependence Plot (PDP) shows the marginal effect of one or two features on the predicted outcome of a fitted model. It isolates the relationship between the chosen feature(s) and the target variable, holding all other features constant (or averaging their effects). For

'tenure', this plot will illustrate how changes in a customer's tenure impact the predicted probability of churn by our tuned Random Forest model.

```
from sklearn.inspection import PartialDependenceDisplay

plt.figure(figsize=(10, 6))
# The `preprocessor` step needs to be handled implicitly or explicitly
# For simplicity, we'll plot directly from the pipeline for a single
feature.
# Note: `PartialDependenceDisplay.from_estimator` automatically
applies the preprocessor if the estimator is a pipeline.

PartialDependenceDisplay.from_estimator(
    best_rf_model, X_train, ['tenure'],
    # feature_names=all_feature_names_rf, # Removed to allow inference
from X_train
    # target=1, # Removed, defaults to positive class for binary
classifiers
    ax=plt.gca() # Use the current axes
)

plt.title('Partial Dependence Plot for Tenure on Churn Probability
(Tuned Random Forest)')
plt.xlabel('Tenure (months)')
plt.ylabel('Partial Dependence (Predicted Probability of Churn)')
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```




```
performance_data_updated = {
    'Metric': ['Accuracy', 'Precision', 'Recall', 'F1-Score', 'ROC
AUC'],
    'Logistic Regression (Tuned)': [accuracy_tuned, precision_tuned,
recall_tuned, f1_tuned, roc_auc_tuned],
    'Random Forest (Tuned)': [accuracy_rf_tuned, precision_rf_tuned,
recall_rf_tuned, f1_rf_tuned, roc_auc_rf_tuned]
}

performance_df_updated = pd.DataFrame(performance_data_updated)
print("Updated Model Performance Comparison:")
display(performance_df_updated)
```

```
Updated Model Performance Comparison:
```

{"summary":"{\n \"name\": \"performance_df_updated\",\n \"rows\": 5,\n \"fields\": [\n {\n \"column\": \"Metric\",\n \"properties\": {\n \"dtype\": \"string\",\n \"num_unique_values\": 5,\n \"samples\": [\n \"Precision\",\n \"ROC AUC\",\n \"Recall\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Logistic Regression (Tuned)\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0.14219918626392353,\n \"min\": 0.5080213903743316,\n \"max\": 0.8387739285437495,\n \"num_unique_values\": 5,\n \"samples\": [\n 0.6551724137931034,\n 0.8387739285437495,\n

0.5080213903743316\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"Random Forest (Tuned)\",\n   \"properties\": {\n
\"dtype\": \"number\",\n   \"std\": 0.1386368673251929,\n
\"min\": 0.516042780748663,\n   \"max\": 0.8393061045234959,\n
\"num_unique_values\": 5,\n   \"samples\": [\n
0.6843971631205674,\n   0.8393061045234959,\n
0.516042780748663\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   }\n]\
n}","type":"dataframe","variable_name":"performance_df_updated"}

```
display(performance_df_updated)
```

{"summary":"{\n \"name\": \"performance_df_updated\",\n \"rows\":
5,\n \"fields\": [\n   {\n   \"column\": \"Metric\",\n
\"properties\": {\n   \"dtype\": \"string\",\n
\"num_unique_values\": 5,\n   \"samples\": [\n
\"Precision\",\n   \"ROC AUC\",\n   \"Recall\"\
n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"Logistic Regression (Tuned)\",\n   \"properties\": {\n
\"dtype\": \"number\",\n   \"std\": 0.14219918626392353,\n
\"min\": 0.5080213903743316,\n   \"max\": 0.8387739285437495,\n
\"num_unique_values\": 5,\n   \"samples\": [\n
0.6551724137931034,\n   0.8387739285437495,\n
0.5080213903743316\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"Random Forest (Tuned)\",\n   \"properties\": {\n
\"dtype\": \"number\",\n   \"std\": 0.1386368673251929,\n
\"min\": 0.516042780748663,\n   \"max\": 0.8393061045234959,\n
\"num_unique_values\": 5,\n   \"samples\": [\n
0.6843971631205674,\n   0.8393061045234959,\n
0.516042780748663\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   }\n]\
n}","type":"dataframe","variable_name":"performance_df_updated"}

```
import joblib

# Define the filename for the model
model_filename = 'tuned_random_forest_model.joblib'

# Save the best Random Forest model
joblib.dump(best_rf_model, model_filename)

print(f"Tuned Random Forest model saved successfully as
{model_filename}")
```

```
Tuned Random Forest model saved successfully as
tuned_random_forest_model.joblib
```

Comparison of Model Performance

Let's compare the performance metrics of the Logistic Regression model (tuned) and the newly trained Random Forest Classifier.

```
performance_data = {
    'Metric': ['Accuracy', 'Precision', 'Recall', 'F1-Score', 'ROC
AUC'],
    'Logistic Regression (Tuned)': [accuracy_tuned, precision_tuned,
recall_tuned, f1_tuned, roc_auc_tuned],
    'Random Forest': [accuracy_rf, precision_rf, recall_rf, f1_rf,
roc_auc_rf]
}

performance_df = pd.DataFrame(performance_data)
print("Model Performance Comparison:")
display(performance_df)
```

```
Model Performance Comparison:
```


```
{"summary":"{\n  \"name\": \"performance_df\",\n  \"rows\": 5,\n
\"fields\": [\n  {\n  \"column\": \"Metric\",\n
\"properties\": {\n  \"dtype\": \"string\",\n
\"num_unique_values\": 5,\n  \"samples\": [\n
\"Precision\",\n  \"ROC AUC\",\n  \"Recall\"\
n  ],\n  \"semantic_type\": \"\",\n
\"description\": \"\"\n  }\n  },\n  {\n  \"column\":
\"Logistic Regression (Tuned)\",\n  \"properties\": {\n
\"dtype\": \"number\",\n  \"std\": 0.14219918626392353,\n
\"min\": 0.5080213903743316,\n  \"max\": 0.8387739285437495,\n
\"num_unique_values\": 5,\n  \"samples\": [\n
0.6551724137931034,\n  0.8387739285437495,\n
0.5080213903743316\n  ],\n  \"semantic_type\": \"\",\n
\"description\": \"\"\n  }\n  },\n  {\n  \"column\":
\"Random Forest\",\n  \"properties\": {\n  \"dtype\":
\"number\",\n  \"std\": 0.14999543317570396,\n  \"min\":
0.47593582887700536,\n  \"max\": 0.8163657030664703,\n
\"num_unique_values\": 5,\n  \"samples\": [\n
0.6033898305084746,\n  0.8163657030664703,\n
0.47593582887700536\n  ],\n  \"semantic_type\": \"\",\n
\"description\": \"\"\n  }\n  }\n  ]\
n}","type":"dataframe","variable_name":"performance_df"}
```